Fine-Tuning Open LLMs for Complex Information Extraction in Low-Resource Settings: Building Elite Networks from Biographies

Anh-Tu Ngo

University of Iowa

September 2026

Abstract: Recent advances in large language models (LLMs) offer new opportunities for political scientists to transform large corpora of texts into structured data that enables substantively interesting analyses. Nevertheless, many important questions in political science require complex information that is relational, hierarchical, and temporally dynamic: who the actors are (individuals, groups, or states), what attributes they have, what actions they take, which events they are involved in, and how they relate to one another over time. Extracting such information goes beyond classification of documents or estimation of their semantics and remains challenging even for powerful LLMs. This creates a cycle of data scarcity: the lack of structured data motivates the creation of automated extraction tools, yet training these tools require the very same data that are lacking. This paper introduces a codebook-to-JSON pipeline for structured information extraction (IE) in such low-resource settings – converting a codebook into a nested output template, hand-annotating a few hundred documents, augmenting them with synthetic examples, fine-tuning open LLMs for extraction tasks, and evaluating their outputs. Applied to biographies of cabinet elites in Vietnam, Russia, and Cambodia, this pipeline produced a finetuned Mistral Small 3.2 (24B) that matches or even outperforms some of the most powerful proprietary LLMs (GPT 5.6 Luna and Sol and Gemini 3.7 Flash). The finetuned model can reliably extract nested and complicated career records with few missing or hallucinated entries, enabling the construction and analysis of elite networks across countries and years. Although demonstrated on elite biographies, the pipeline offers a generalizable IE strategy for building structured political datasets from complex texts where labeled data is scarce.

Introduction

Converting unstructured texts into structured datasets based on coding rules has been central to many data collection endeavors that empower quantitative social science (Benoit et al. 2025; King and Lowe 2003; Salehyan 2015; Stuhler, Ton, and Ollion 2025).[1] Since such databases often involve enormous amounts of manual labor, scholars have attempted to automate data collection efforts for decades (King and Lowe 2003; Laurer et al. 2024) – an effort that recent advancements in large language models (LLMs) have recently made far more feasible. Most applications, however, extract document-level information – such as classifying or predicting overall features of a document such as topics (Benoit et al. 2025; Brandt et al. 2026; Burnham et al. 2026; Erhard et al. 2025; Y. Hu et al. 2022; Laurer et al. 2024). More recently, generative LLMs have been shown to conduct more detailed data collection – being able to read codebook-derived instructions as prompts, read a document, and fill in a codebook-like template in a single step – essentially turning a codebook into a "promptbook" (Stuhler, Ton, and Ollion 2025).

Nevertheless, existing demonstrations have mostly focused on relatively flat data templates, in which each variable is recorded as a single value or a simple list. Few studies have shown whether LLMs can actually perform the kind of extraction that many political science data collection processes require: long documents, outputs with dozens of nested entries, and relational details that require some level of logical inference. In other words, the tasks that would be most valuable to automate are also the most complicated ones.

This problem is particularly apparent in the study of political elites, where scholars have long sought to understand elites' networks and how they shape many policies and political outcomes. Because ties among elites are rarely observed directly, they are commonly inferred from overlaps in elites' careers, education, and families (Baturo and Elkink 2016; Keller 2016; Meyer, Shih, and Lee 2016). This requires fine-grained records of where and when each elite studied and worked. For example, a political elite's career often contains many episodes, each of which includes a position attached to a specific department within an organization, based in a particular location, and during a particular period, where she may interact with many other individuals and organizations – all of which are scattered across a long biography. To complicate the matter, names of people, organizations, departments, locations, and other entities can be recorded under multiple names or change their names over time. A missing or misplaced detail is therefore not a trivial error: dropping a department or attaching a date to the wrong position can create ties that never existed and overlooks ones that did. Hand-coding such records from long biographies is extremely costly, which has largely restricted existing elite network datasets to a single country or a specific type of elite (Baturo and Elkink 2016; Keller 2016; Meyer, Shih, and Lee 2016).

[1] The collection of sources and extraction of information can be found in many popular political science datasets such as the V-Dem Project (Coppedge et al. 2026), Uppsala Conflict Data Program (UCDP) (Davies, Pettersson, and Öberg 2026; Gleditsch et al. 2002; Harbom, Melander, and Wallensteen 2008), or Issue Correlates of War (ICOW) (Hensel 2001; Hensel and Mitchell 2015).

Existing tools are not optimally suited for this task. Traditional systems of information extraction (IE) – a family of natural language processing (NLP) methods for identifying entities, relations, and events (Xu et al. 2024; Zhang et al. 2025)[2] – often require researchers to train several specialized models, each for a single sub-task, and break a document down into isolated pairs or triplets of entities and relations. This is not intuitive to the codebook-based records that political scientists usually produce (Stuhler, Ton, and Ollion 2025). On the other hand, prompting general-purpose LLMs does not guarantee high-quality extractions where each detail ends up in the right entry. Doing so reliably requires a model to shift its behavior – attending to details across a long context, making (limited) logical inferences, and strictly adhering to a complex output template. This calls for fine-tuning on curated data.

Nevertheless, training data are the most lacking precisely for these complicated tasks. This results in a circular scarcity problem: the lack of structured elite data motivates the development of computational tools to help with large-scale data collection yet also inhibits that very same development. Fine-tuning moderate-size open LLMs is a promising solution that allows smaller models to rival or even outperform much larger commercial models, while being cheaper to run at scale (Burnham et al. 2026; Z. Hu et al. 2023; Zhao et al. 2024). Nevertheless, much of this evidence is presented for classification or other tasks with relatively short or flat (non-nested) outputs and utilizes training datasets with at least thousands of examples – larger than what many social scientists in data-scarce situations can afford to annotate by hand. Whether fine-tuning can deliver similar gains when the task is complicated, the codebook is detailed, *and* the training examples are modest in number remains uncertain.

This paper addresses this gap by introducing and evaluating a pipeline to fine-tune moderate-size open LLMs for complex, structured IE tasks in low-resource settings. The pipeline consists of five steps: (1) translating a codebook into a nested output template (a JSON template)[3], (2) manually annotating a few hundred documents, (3) supplementing them with synthetic examples generated by a larger "teacher" model, (4) fine-tuning moderate-size open LLMs with parameter-efficient methods, and (5) evaluating the outputs against human-annotated records, piece by piece. The resulting model converts a long, unstructured document into a structured record that is readily usable for analysis – all in a single step.

I demonstrate this pipeline on the information extraction of biographies of elites in the government cabinets from Vietnam, Russia, and Cambodia. Specifically, with only 445 main training examples and 350 short synthetic examples, I fine-tune two moderate-size open LLMs –

[2] They are also called named entity recognition (NER), relation extraction (RE), and events extraction (EE).

[3] JSON (JavaScript Object Notation) is a widely used, text-based format for storing data in a tree-like structure. Each piece of information is stored under a label (a "key") – for example, "Name": "Viktor Yerin" – and a label can contain not only a single value but also a list of entries, each with its own set of labels. This allows JSON to preserve the nested relationships among details – which position belongs to which organization, and during which period – exactly as they appear in a biography. JSON records can also be easily converted back into conventional data tables (e.g., one row per job) for statistical or network analyses. This is what makes JSON particularly suitable for political texts.

NuExtract 2.0 (8B) and Mistral Small 3.2 (24B) to read each elite's biography and generate a nested structured JSON record containing her education, career history, major events, family ties, and professional allies/rivals. I then compared these models against three frontier proprietary LLMs (GPT 5.6-Luna, GPT 5.6-Sol, and Gemini 3.7 Flash) and demonstrate how the extracted JSON records can then enable the detection of overlaps among elites' educational, career, and familial histories that results in elite networks across countries and years.

The results show the promises of this approach, while also raise important considerations. First, the task is not easy even for frontier models with massive sizes, immense knowledge bases, and strong reasoning capabilities: all five model setups can identify the right content better than they place it in the right entries. Second, the finetuned Mistral Small 3.2 (24B) outperforms both GPT 5.6 models and performs on par with Gemini 3.7 Flash, while the much smaller NuExtract (8B) performs on par with the two GPT 5.6 models. Third, the finetuned Mistral is able to extracts the fields that directly serve the construction of elite networks – particularly jobs and education – with few missing or invented entries. Furthermore, the models tend to be conservative, preferring not to extract uncertain information more often than making up new details. This implies that networks built from their outputs are more likely to miss true ties than to contain false ones. On the other hand, rare fields that require more inference (such as dismissals, resignations, and rivalries) remain challenging for all models. Finally, a network of Vietnam's cabinet in 2000, constructed from the finetuned model's (human-corrected) outputs, reflects the commonly known factional dynamics as reported in historical narratives.

Through this pipeline, this paper makes four contributions. First, it demonstrates that moderate-size open LLMs (8B-24B parameters) that can reliably conduct complex IE tasks simultaneously, matching or even outperforming much larger general models. Second, this work also shows how extreme data scarcity (especially for complex tasks) can still be mitigated with a combination of human-annotated and synthetic data. Third, I introduce a comprehensive codebook (and output JSON template) that push the LLMs to not only extract fine-grained details about political elites but also normalize various names of people, institutions, and titles into a standardized format – thus simplifying the co-reference/entity linking problem, allowing multiple steps of IE to be done in one single step, and enabling large-scale data collection of elite data. Last but not least, this end-to-end IE approach can be adapted and generalized to any research problem requiring detailed information of actors, their attributes, relations, and events – thus opening many substantively interesting research avenues that has been previously difficult to pursue.

In the following sections, the paper discusses why elite networks require fine-grained and nested records, followed by a review of existing approaches to information extraction, along with the technical challenges of LLM-based structured IE. The paper then describes my approach, fine-tuning setups and evaluation of the models. The paper then examines the surface validity of this approach by a qualitative analysis of an elite network in Vietnam resulted from the Mistral's extracted JSONs. Finally, the conclusion section provides a summary of the approach, its implications, and future research paths.

Elite Networks and the Need for Nested Extraction

Elites remain some of the most important actors shaping socio-economic, political, and policy outcomes across the world. Yet, systematic data on elites – especially their social networks and interactions – are limited. This is not surprising: because of many elites' inaccessibility to the public, interviewing or surveying elites is costly and carry risks of incomplete or misrepresented information. Recent observational datasets – such as WhoGov (Nyrup and Bramwell 2020), Global Leadership Project (GLP) (Gerring et al. 2019), and Paths to Power (Nyrup et al. 2024) – offer impressive coverage of cabinet elites across countries and years. However, they mostly contain broad background variables of elites (such as the elites' broad occupation categories or their families' socioeconomic background) rather than relationships. The few datasets that do enable network analysis are often limited to one country or a specific elite population, such as Chinese Communist Party members (Meyer, Shih, and Lee 2016), North Korea's ruling elites (Ishiyama and Kim 2020; Kim and Ishiyama 2023), Russian patronage networks (Baturo and Elkink 2016), and Eastern Europe's Politburo members (Matthews 2023).

Building elite networks from biographical data generally follows one of three approaches, each placing different demands on the source texts (Table 1).

Table 1. Approaches to Constructing Elite Ties from Biographical Data

Approach	Example of a tie	Information required	Extraction complexity	Role of LLMs
Shared background attributes	Both elites were lawyers before entering politics	One value or a short list per elite (occupation)	Non-nested; often already in cross-national datasets	Increase speed
Reported Ties	A biography or news report names another elite as a friend or ally	Names of associates in the text	Non-nested: recognize & match person names	Increase speed
Overlapping career & educational episodes	Two elites worked in (1) the same department, (2) of the same organization, (3) at positions close to each other (e.g., head and deputy head of department), (4) in the same years	Position, organization, department and/or smaller unit, location, and dates – jointly, for every episode	Nested: up to dozens of educational/career episodes per elite, each with linked subfields; names need to be standardized across documents to enable matching & tie detection	Necessary: one LLM must read the whole biography, connect each detail to the right entry/episode, and standardize names

The first two approaches are relatively easy to automate, but both carry higher risks for network measurements. The first approach rests on strong assumption that two elites may know each other or even share political interests simply because they used to be doctors or lawyers, while the second approach tends to collect data on only a handful of high-profile elites.

On the other hand, the third approach allows much safer assumptions about how ties form and has become the established convention in studies of elite factional politics (Keller 2016; Meyer, Shih, and Lee 2016; Shih, Adolph, and Liu 2012). Of course, it also requires an enormous amount of data on the specific schools and program of studies, organizations, subunits, locations, and timeframes of each elite – organized in hierarchical ways. Following established conventions, I detect network ties based on how closely two elites worked together within the same administrative and organizational hierarchy during the same period – that is, whether (1) they worked in the same body at position ranks close to one another (e.g., head and deputy head) of the same organization, (2) one elite led an organization while the other led a subunit within it, or (3) both led subunits under the same parent organization. Applying such rules requires each career episode to be extracted as a single, connected record: the organization, the department and/or unit nested within it, the position, the location, and the period. Preserving such records across dozens of episodes per elite, while normalizing the many names of the same organizations or position titles, is precisely the task that the models in this paper are fine-tuned to perform.

Approaches to Information Extraction

Information Extraction (IE) as a Family of Tasks

Information extraction is a family of NLP tasks that aim to automate the identification and extraction of key details from documents. They are traditionally done in multiple phases (figure 2).

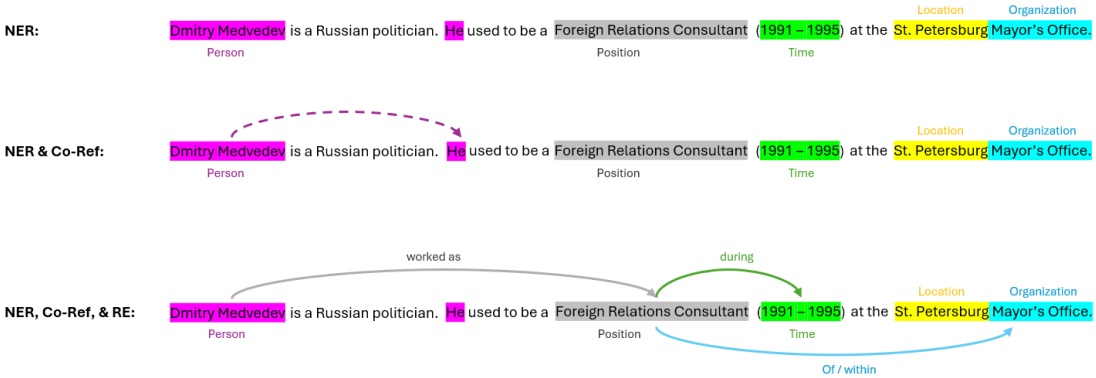


Figure 2. An example of Information Extraction tasks (NER, Co-Ref, and RE)

An IE system usually begins with **named entity recognition** (NER), which recognizes and classifies different types of entities – in the context of this project, persons, job titles, organizations, sub-organizations, dates, and locations. Next, **co-reference resolution** (Co-Ref) links the different mentions of the same entity such as "Mr. Medvedev", "he", or "Medvedev" (similar to Communist Party of Vietnam being referred to as "the Party" or "CPV"). Finally, **relation extraction** (RE) identifies the relationships among the recognized entities. Without RE, a model may understand a sentence in figure 2 as a collection of unconnected entities – "Dmitry Medvedev," "Foreign Relations Consultant," "St. Petersburg," "Mayor's Office," and "1990–1995." With RE, the system

can treat relevant entities as a connected chain and even detect person-to-person relations or multi-entity relations in key events (e.g., who sacks or purges whom from which position).

Traditional Approaches

The conventional approach to IE comprises multiple steps (figure 3A), each handled by a specialized token-classification models, with the output of one model becoming the input of the next (Stepanov and Shtopko 2024; Zaratiana et al. 2024). The final output of such an IE system often comes in the form of a knowledge graph, which represents the extracted details as triplets or quadlets of entities (nodes) and relations (edges) – for example, [Elite A] <studied at> [University X] and [University X] <located at> [Location Z].

As previously noted, this approach is difficult to apply to social science research. First, it requires researchers to have technical familiarity to not only use but train multiple models for specific tasks (each with its own training data). Second, its outputs of pairwise and triplet relations treat entities and relations as independent and equal, which neither represent the hierarchical interdependencies of actors and events in politics nor align with how social scientists collect their data, which often involves reading a document and converting relevant information into a structured data format (such as a data table) guided by a codebook (Stuhler, Ton, and Ollion 2025).

LLM-Based Approaches

Recent developments in LLMs have shifted the framing of IE from a token-classification to a sequence-to sequence (seq2seq) problem (Constantin et al. 2025; Wu et al. 2024; D. Xu et al. 2024; Zhang et al. 2025). Specifically, rather than focusing on tagging the specific words/tokens in a document, LLMs can read a document as a sequence and regenerate a "summary" sequence that contains all relevant information in a structured way.

This offers two major benefits. First, as demonstrated by figure 3B, the output can be a JSON record that contains complex and hierarchical information in a tree-like structure – rather than breaking them down into triplets – making it readily usable for network construction with minimal processing. Second, since a single LLM can perform multiple IE subtasks at once, the entire process can be unified with just one extractor with one set of model weights, thereby saving the time and resources needed to train (and annotate data for) separate models.

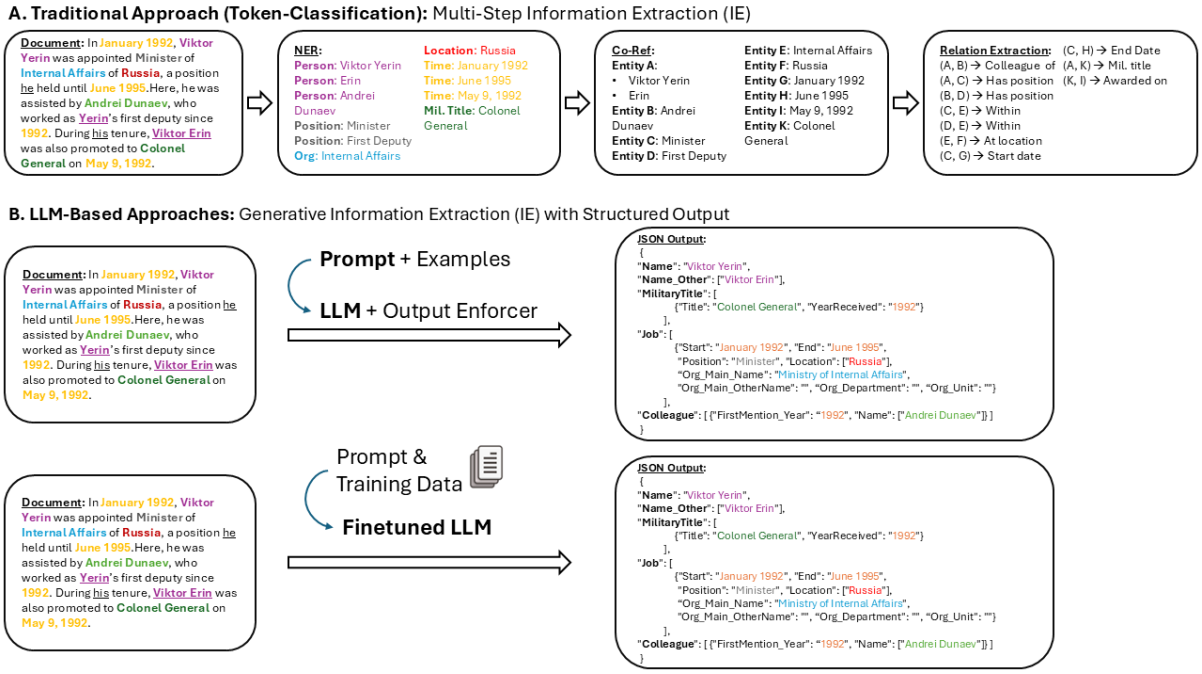


Figure 3. Generative LLM vs. Traditional AI for Information Extraction

Importantly, this generative approach also brings IE much closer to how social scientists have traditionally coded their data. Researchers can describe their coding rules in natural language and ask the model to apply them – essentially turning a codebook into a "promptbook" (Stuhler, Ton, and Ollion 2025). Specifically, Stuhler et al. demonstrates that, with prompting, open LLMs can extract a variety of biographical details (such as age, occupation, and origin) from newspaper obituaries with high accuracy. Like most existing applications, however, their templates are relatively flat (non-nested): each variable is recorded as a single value or a simple list. The robustness of this approach when a researcher needs to extract JSON records containing dozens of nested entries – each with its own dates, locations, and organizational units – is unclear.

Technical Challenges of LLM-based Structured IE

Even with their notable capabilities, LLMs may still find it challenging to deal with the duo-task of (1) extracting accurate information while (2) organizing them into valid structured outputs (Li, Ramprasad, and Zhang 2024; Y. Liu et al. 2024). Prompting techniques (Li, Ramprasad, and Zhang 2024) and output constraints (such as Outlines or Instructor)[4] can force LLMs to generate structured JSONs based on user-defined templates and be helpful for simple, non-nested JSON outputs. However, it may not be optimal for the type of political-biographical data in this paper.

[4] Recently, more and more LLMs (including model families like GPT, Claude, or Mistral) have included natively built-in capabilities to generate structured output (most commonly JSON mode) without the need for external tools like Instructor or Outlines. However, these native capabilities cannot guarantee 100% valid JSONs

The second challenge – the scarcity of data – follows suit. Most NLP datasets for structured IE are designed to train/finetune models for single specific tasks such as NER and RE (Cheung et al. 2024; Huguet Cabot and Navigli 2021; Tjong Kim Sang and De Meulder 2003), while popular models specifically trained for structured IE, such as GLiNER and NuExtract tend to be geared toward non-nested or lightly nested extractions (Constantin et al. 2025; Zaratiana et al. 2024).

Given these factors, to extract long, nested, and interdependent details from political texts, a LLM needs to systematically shift its behavior, emphasizing document-level comprehension, detailed recall, logical reasoning, and adherence to complex output templates. Such a behavioral shift requires fine-tuning the model on curated data, rather than relying on prompting techniques and external output enforcers alone.

A Codebook-to-JSON Pipeline

The pipeline consists of five steps. While each step involves several technical choices, the overall process aligns with how political scientists have traditionally built datasets by hand: defining the concepts and coding rules, coding a sample of documents, training the coders, and checking the quality of their work. The key difference is that the "coder" here is a finetuned LLM. This section describes the first four steps, while the fifth step – evaluation – is the focus of the next section.

Step 1: From Codebook to Output Template

In this project, the output of the collected data is stored in JSON records. The output follows a JSON template with 18 fields (figure 4), each corresponding to a variable in a conventional codebook. Table 2 summarizes these fields, their nested subfields, the types of tasks they require, and their key coding rules. Unlike a flat (non-nested) data table, the template allows a single elite's record to contain dozens of entries for the same field (e.g., every job she has held), with each entry containing its own subfields (e.g., the position, organization, department, location, and dates of that job). In other words, the template preserves the hierarchical relationships among the details in a biography, rather than breaking them down into separate variables or triplets.

```
{
"Name": "",
"Name_Other": [],
"BirthYear": "",
"DeathYear": "",
"Birthplace":
    {"City": "", "City_OtherName": [], "Region": "", "Region_OtherName": []},
"Ethnicity": [],
"Education": [
    {"School": "", "OtherName": [], "Start": "", "End": "",
     "Level": "", "Field": "", "Location": []}
  ],
"MilitaryTitle": [
    {"Title": "", "YearReceived": ""}
  ],
"Job": [
    {"Start": "", "End": "", "Position": "", "Location": [],
     "Org_Main_Name": "", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""}
  ],
"Retired": [
    {"Date": "", "Detail": []}
  ],
"Dismissed": [
    {"Date": "", "Detail": []}
  ],
"Resigned": [
    {"Date": "", "Detail": []}
  ],
"FamilyMember": [
    {"Name": "", "Relation": ""}
  ],
"Colleague": [
    {"FirstMention_Year": "", "Name": []}
  ],
"Rival": [
    {"FirstMention_Year": "", "Name": []}
  ]
}
```

Figure 4. Output JSON Template for Structured IE

The coding rules serve two purposes. First, they define what counts as each piece of information – for example, what distinguishes a resignation from a dismissal, or a rival from a mere critic. Second, they normalize the extracted information into standardized formats. For instance, the rules require dates to be recorded as "Month YYYY" or "YYYY," and the jurisdiction in a government body's name to be moved into the Location subfield (so that the "Ministry of Internal Affairs of Russia" is recorded as the "Ministry of Internal Affairs" located in "Russia"; see figure 3B). Such normalization makes it much easier to detect whether two elites worked in the same organization – essentially folding part of the co-reference problem into the extraction step itself.

Next, the coding rules are turned into prompts – or, following Stuhler, Ton, and Ollion (2025), a promptbook. For the evaluation section, two prompt versions are used: a shorter version for the finetuned models (since they also learn the rules during training) and a longer version with mode detailed rules for the commercial models (since they cannot be finetuned). Both prompts can be found in **appendix 10**.

Table 2. Summary of the Codebook (Corresponding to Output JSON Template Fields)

Section	Field	Subfields (per entry)	Task type*	Key coding rules (shortened)	Example
Basic information	Name	--- (single value)	Extraction; normalization	Principal standardized (romanized) name of the elite.	"Viktor Yerin"
	Name_Other	--- (list)	Extraction	Aliases, birth or married names, alternate transliterations, and native-script forms.	["Viktor Erin"]; ["Олег Дмитриевич Давыдов"]
	BirthYear, DeathYear	--- (single value)	Extraction; normalization	Year only; omitted if sources conflict and the biography does not resolve the conflict.	"born on 25 May 1940" → "1940"
	Birthplace	City, Region (and their other names)	Extraction; normalization	Most specific named place as City; first-level administrative unit as Region.	"born in Yekaterina, Sverdlovsk oblast" → City: "Yekaterina", Region: "Sverdlovsk"
	Ethnicity	--- (list)	Extraction	Only if explicitly stated for the elite or her parents; never inferred from names, birthplace, or language.	---
Education and career	Education	School, OtherName, Start, End, Level, Field, Location	Extraction; normalization; placement	Formal schooling, academies, and training programs.	School: "Moscow Institute of Civil Engineering"; OtherName: ["MISI"]; End: "1963"; Level: "University"; Field: "Engineering"; Location: ["Moscow"]
	MilitaryTitle	Title, YearReceived	Extraction	Formal military (or equivalent) ranks only.	"promoted to Colonel General on May 9, 1992" → Title: "Colonel General"; YearReceived: "1992"
	Job	Start, End, Position, Location, Org_Main_Name, Org_Main_OtherName, Org_Department, Org_Unit	Extraction; normalization; placement	Formal positions only; separate entries for concurrent positions; jurisdictions moved from government bodies' names to Location.	"From 1992, he served as Minister of Internal Affairs of Russia until 1995" → Start: "1992"; End: "1995"; Position: "Minister"; Org_Main_Name: "Ministry of Internal Affairs"; Location: ["Russia"] Org_Department: "--"

Career events	Retired, Resigned, Dismissed	Date, Detail	Classification; inference; verbatim extraction	**Retirement** ends an entire career; **resignation** is a (possibly pressured) departure framed as such; **dismissal** is a forced removal (incl. purges and coups); Detail copies the supporting sentences.	"On 30 June 1995, after the failed operation to free hostages taken by Chechen terrorists in Budyonnovsk, Yerin had to resign (as did the then-FSB director Stepashin)" → Resigned: {Date: "June 1995"; Detail: "…exact-sentence…"}
	Arrested, Exiled, Killed	Date, Detail	Classification; inference; verbatim extraction	Actual detention (not mere investigation); forced exile (not voluntary emigration); intentional killing (not natural death or accident).	
Relationships	FamilyMember	Name, Relation	Extraction; inference	Named relatives only; most specific relation, with the correct direction of the relationship.	"He was married to Lyubov Leonidovna Erina (1940-2025) and had two children: son Leonid (born 1971) (officer, serves in the Federal Security Service of Russia) & daughter Nadezhda. → {Name: "Lyubov Leonidovna Erina"; Relation: "Wife"}, {Name: "Leonid Yerin"; Relation: "Son"}
	Colleague	FirstMention_Year, Name (list)	Inference; extraction	Explicit, meaningful professional association (collaboration, mentorship, alliance); *not* mere co-occurrence or succession.	"…assisted by Andrei Dunaev, … first deputy since 1992" → {FirstMention_Year: "1992", Name: ["Andrei Dunaev"]}
	Rival	FirstMention_Year, Name (list)	Inference; extraction	Explicit political or power struggle (factional conflict, purge, coup); *not* mere criticism, policy disagreement, or electoral competition.	
All fields	(general rules)			- Use only information stated or directly supported by the text; - Prefer omission over speculation; dates as "Month YYYY," "YYYY," or "Present"; no duplicated entries.	

Note*: **Extraction = locating information stated in the text; **normalization** = converting it into a standardized format; **placement** = assigning each detail to the correct entry and subfield; **classification** = choosing among predefined event types; **inference** = judging whether textual cues meet a definition (e.g., what counts as a rival). The full codebook and real example JSON output is presented in **appendix 9**.

Steps 2 and 3: Human-Annotated and Synthetic Training Data

The second step is to hand-annotate a set of documents using the codebook. While presented as a separate step, annotation often goes hand in hand with codebook development: annotating real documents often reveals cases that the initial rules do not cover, which in turn leads to clearer rules. In this project, biographies of cabinet elites in Vietnam, Russia, and Cambodia (in their native language) are downloaded from Wikipedia via the WikiAPI and translated into English with the Google Translate API. I then select a number of these biographies (mostly of Vietnamese and Russian elites), identify gaps and/or conflicting information in each biography, and manually enrich them using other verified internet sources before annotating their JSON records.

The third step supplements these human-annotated examples with synthetic ones, generated by a larger teacher model (GPT-4.1) and then manually corrected. The use of synthetic data (biographies) serves two purposes: it (1) significantly increases the number of training examples without costing too time and resources and (2) increase the diversity of the data that the model learns from – given the diverse styles, lengths, and entity types of LLM-generated documents. Synthetic examples as a data augmentation strategy has been shown to significantly increase model performance, especially for IE tasks (Ye et al. 2024).

I generate synthetic data in three ways. First, I reverse-generate biographies from the manually annotated JSON records, producing paraphrased (but still factual) versions of the original biographies that contain the exact details in the records. Reverse synthetic data generation is particularly suitable for complex tasks (such as IE), where the LLM needs to learn the precise patterns of extraction (Josifoski et al. 2023). Second, I created fully synthetic JSON records by randomly swapping the names of people, organizations, locations, and dates in the hand-coded JSONs for artificial ones, and then reverse generate synthetic biographies from these synthetic JSONs. Third, I also create shorter JSONs and biographies (of only one to three paragraphs) that only contain information about rare fields (Dismissed, Resigned, Retired, Colleague, and Rival).

Table 3. Summary of Training, Validation, and Test Data

	Set	Train	Validation	Test	Total
Main Data	Original	207	16	39	262
	Paraphrased bio (factual)	158	12	30	200
	Fully synthetic	80	6	16	102
Supplement Data	Fully synthetic (short)	350	0	0	350

As table 3 shows, the *main data* includes 564 examples: 262 original (human-annotated) examples, 200 factual examples with paraphrased biographies, and 102 fully synthetic examples. This dataset is split into the training set (445 examples), validation set (34 examples), and test set (85 examples). The 350 short synthetic examples are used for training only. The validation set is used as in-training evaluation, allowing me to choose the best version of the fine-tuned models

before final testing, while the test set is used to evaluate the model's performance after fine-tuning (using documents they have never seen). Importantly, *all versions of the same elite's biography are assigned to the same data split, so that the models are never tested on a paraphrase of a biography that they saw during training*.

Steps 4: Fine-Tuning

The fourth step fine-tunes moderate-size open LLMs on the annotated examples. Specifically, I rely on *supervised fine-tuning (SFT)*, a process where an already pre-trained LLM learns from additional data – pairs of input (biographies) and desirable/expected output (JSON records; see figure 3B) – curated for a specific task. This keeps the model's foundational knowledge learned during pre-training while updating its internal weights[5] to improve its structured IE performance.

Rather than updating all of the model's parameters or weights (*full fine-tuning*), which requires substantial computing resources and large annotated datasets as well as risks overfitting or even *catastrophic forgetting* (Han et al. 2024, 15; Kirkpatrick et al. 2017; Luo et al. 2025; L. Xu et al. 2023, 1), I use *parameter-efficient fine-tuning (PEFT)*. PEFT only updates a small subset of the pretrained model's weight structure and thus requires a smaller task-specific dataset and less computing resources – in addition to reducing the risk of overfitting (Z. Hu et al. 2023; Luo et al. 2025; L. Xu et al. 2023). This makes PEFT less risky and more practical in data-scarce situations.

Among many PEFT techniques, I choose the *low-rank adaptation (LoRA)*. Essentially, LoRA freezes the LLM's original parameters, injects small adapters (low-rank matrices) into each layer within the LLM, and trains only those adapters[6] (Hu et al. 2021; Hu et al. 2023; Xu et al. 2023). Despite updating a small number of parameters, LoRA is very effective: models finetuned with LoRA not only exceed the non-finetuned models' performances by an average of 38.7% but sometimes can outperform GPT-4 by an average of 2.7% (Zhao et al. 2024).

Model Choice

There are two major reasons driving my choice of models to finetune: innate capabilities and computational costs. On the one hand, the chosen LLMs should have some inherent ability to (1) understand long and complex documents, (2) make logical reasoning and inference, and (3) adhere to generate structured outputs. On the other hand, the chosen LLMs' should be moderate in size since models of over 30B parameters are not trainable and usable with limited GPU resources.

Based on these considerations, I choose NuExtract 2.0 (8B) and Mistral Small 3.2 (24B) as key models to fine-tune.[7] NuExtract 2.0 is a fully finetuned (using the company NuMind's

[5] Hence an internal and permanent shift in behavior toward a specific task.
[6] More details about LoRA can be found in Hu et al. (2021) and Zhao et al. (2024).
[7] 8B and 24B parameters mean that the models have 8 and 24 billion parameters, respectively. Each parameter represents an internal numerical variable (weights and biases) that are adjustable based on the training/finetuning data. LLM parameters are mathematically similar to parameters in regression models.

proprietary dataset) from the base model Qwen 2.5-VL for information extraction, outperforming even frontier LLMs at the time (GPT-4.1 and Claude 4 Opus) at structured IE tasks (Constantin et al. 2025). Mistral Small 3.2 (24B), on the other hand, is already trained to follow instructions and generate structured JSON outputs (Mistral AI 2025), while its larger size of 24B offers a larger knowledge base and stronger reasoning capability – all of which enables it to not only follow my extraction rules but also make necessary logical inferences (Mistral AI 2025).

Finetuning Setup

The details of fine-tuning hyperparameters are presented in table 4. For NuExtract (8B), a smaller model that has already been trained for structured IE tasks, I set the LoRA rank and alpha at lower values (16 and 32, respectively) to ensure the model does not learn too much from the small training set and train it over 3 training epochs with gradually decreasing learning rates.

Table 4. Training Setups for the Main LLMs

	NuExtract 2.0 (8B)	Mistral Small 3.2 (24B)
Training Data	445 (~79% total data)	
Validation Data	34 (6% total data)	
Test Data	85 (15% total data)	
Quantized Model	No	Yes
LoRA Rank	16	64
LoRA Alpha	32	128
Training epochs	2 + 1	2
Learning rate	1e-4	1e-4
Training Steps	100 + 50	100

On the other hand, Mistral is too large for my computational resources in its original form, thus requiring me to fine-tune it with QLoRA – which follows the same logic as LoRA but quantizes (reduces) the model's numerical weights (from 16-bit to 4-bit numbers) before training. Intuitively, quantization is similar to rounding very long decimal numbers to fewer digits so that they take up less space, allowing large LLMs to be trained and used with limited computing resources at the cost of slightly lower accuracy. To compensate for this loss, I allow a higher rank and alpha for Mistral (64 and 128, respectively), while training it for 2 epochs.

Evaluation of Models Performance

After fine-tuning, the models are used to generate predicted JSON extractions, which are compared against the 85 human-validated examples in the test set. I compare the performance of five structured IE pipelines: (1) GPT 5.6-Luna (medium reasoning effort), (2) GPT 5.6-Sol (medium reasoning effort), (3) Gemini 3.7 Flash (medium reasoning), (4) NuExtract (finetuned), and (5) Mistral Small 3.2 (finetuned).

This setup serves two purposes: (1) it compares the fine-tuned models against some of the most powerful commercial LLMs, which also have the added strength of being reasoning models, and (2) it shows how two models of different sizes perform with the same training data. While the GPT and Gemini models cannot be finetuned, I rely on their capability for in-context learning: they receive a longer prompt with more detailed instructions along with nine examples, each including a biography and an output JSON records (human-annotated). The examples are randomly drawn from the training set to represent the three countries (four from Vietnam, three from Russia, and two from Cambodia). Such few-shot prompting has been shown to help powerful LLMs to match or even outperform finetuned models across various tasks (Brown et al. 2020; Lampinen et al. 2025; S. Liu et al. 2024; Oskooei et al. 2026; Trad and Chehab 2025) – with performance peaking around 5-20 examples (Oskooei et al. 2026; Tang et al. 2025).

Evaluation Metrics

Evaluation of the models' performance is done by comparing each model's predicted JSONs against the human-annotated JSON records in the 85-example test set. Finding suitable evaluation metrics for nested outputs is a challenge in itself. Conventional classification metrics are not suited to outputs whose entries can be partially correct, listed in different orders, or split and merged differently from the gold standard. To reveal a holistic picture of the models' extraction quality, I use a set of metrics (table 4) to evaluate both what each model extracts (content) and where it places that information (structural adherence) – both at the overall level (each elite's entire record) and field-level (by each field in the JSON template).[8]

JSON Validity: The first metric is the percentage of outputs that can be read as a valid JSON object – that is, outputs with properly closed braces and brackets, double-quoted keys, and no trailing/redundant commas. Importantly, an invalid JSON is treated as an *empty* prediction – meaning that every piece of information in the corresponding gold-standard JSON counts as missed. In other words, models are penalized for failing to generate readable JSON records. All metrics below are calculated over the $D = 83$ test documents that can be scored.[9]

[8] In this paper, the terms "micro" and "macro" refer to *how* a score is averaged – pooling all extracted items together (micro) or averaging across fields (macro) – rather than to the distinction between overall and field-level metrics.

[9] Two of the 85 test examples are duplicated entries of the same elite with no usable gold-standard record, so they cannot be scored for any model. JSON validity is still calculated over all 85 outputs.

Table 4. Metrics for Structured IE Evaluation

	Metric	Measure
Overall Level	JSON Valid (%)	Share of outputs that are valid (readable) JSON objects
	Avg. BoW Jaccard Similarity	Overlap of unique words between predicted and gold JSONs (structure ignored), averaged across documents
	Micro Lexical F1, Lex-Precision, and Lex-Recall	Accuracy of extraction with partial credit, pooled across *all extracted items*
	Micro Cosine F1[10]	Same as Micro LexF1, but based on meaning (semantic) similarity; serves as robustness check
	Macro Lexical F1[11]	Accuracy of extraction with partial credit, averaged across *all fields* (each field weighted equally)
	Hallucination Rate	Share of predicted items that do not resemble any gold item
	Omission Rate	Share of gold items that do not resemble any predicted item
Field-Level	Avg. BoW Jaccard (figure 6A)	Overlap of unique words *within each field* of the predicted and gold JSONs
	Lexical F1 (figure 6B)	Accuracy of extraction with partial credit within each field
	Hallucination & Omission Rate (appendix 3)	Hallucination & omission rates, by field

Lexicon-Based Metrics and Calculation Procedures

Avg. BoW Jaccard Similarity: The average bag-of-words (BoW) Jaccard measures the overall similarity between the LLM-generated and human-annotated JSON records. As its name suggests, it treats each JSON as a "bag" of words and measures whether the models identify the right names, organizations, dates, and other details – ignoring the JSON structure altogether (whether such content ends up in the right entry or subfield) (see appendix 4). For each document d, the set of unique words that appear in the values of the predicted JSON is T_d^P and of the gold-standard JSON is T_d^G.[12] The Jaccard similarity is then the number of words that the two sets have

[10] Robustness checks; explained in **appendix 5**.
[11] Robustness checks; explained in **appendix 7.**
[12] Words are identified as sequences of letters and numbers (punctuation is dropped) and converted to lowercase. However, accents are not removed. This means that "Hồ Chí Minh" and "Ho Chi Minh" share only one of five unique words (a similarity of 0.2).

in common divided by the total number of unique words across both sets. The average BoW Jaccard is the mean BoW score over the total number of documents ($D = 85$):

$$Jac_d = \frac{\left|T_d^P \cap T_d^G\right|}{\left|T_d^P \cup T_d^G\right|}, \qquad \text{Avg. BoW Jaccard} = \frac{1}{D}\sum_{d=1}^{D} J_d^{BoW} \qquad (1)$$

Items and Item Matching: To measure extraction accuracy more directly, the remaining metrics compare the predicted and gold-standard JSONs at the level of ***items***. An item is a single piece of information that the model is expected to extract. For simple fields like Name, BirthYear, and DeathYear, each field contains only one item (the value itself). For Birthplace, the whole object (city, region, and their alternative names) is one item. For list fields like Education or Job, each entry is an item – an elite with 20 professional experiences therefore has a list of 20 Job items. Similarly, each alias in Name_Other and each group in Ethnicity counts as one item. For Colleague and Rival, each person-year pair is treated as one item, so that a model is not penalized simply for grouping the same colleagues differently.[13]

Comparing two items requires a measure of how similar they are. For a predicted item p and a gold item g, I calculate the item similarity as the *average word-level Jaccard similarity* across the subfields of the two items:

$$s(p, g) = \frac{1}{\left|K_{p,g}\right|}\sum_{k \in K_{p,g}} \frac{|T(p_k) \cap T(g_k)|}{|T(p_k) \cup T(g_k)|} \qquad (2)$$

where $T(\cdot)$ is the set of unique words in a subfield value and $K_{p,g}$ is the set that includes only subfields that are filled in at least one of the two items. Subfields that are empty in both items are left out, since they provide no information about the quality of extraction. A subfield filled on only one side receives a score of 0. As a result, $s(p, g)$ ranges from 0 (two items have nothing in common) to 1 (every filled subfield matches word for word).

Because the model and the human coder may list dozens of jobs in different orders, I then use the *Hungarian matching algorithm* (Kuhn 1955) to pair each predicted item with *at most one gold item* (and vice versa) in a way that maximizes the total similarity between the predicted and gold items in field f of document d. Any leftover items remain unpaired and earn no credit. For instance, when a model extracts 12 jobs while the gold record has 15, there are 3 leftover/unpaired items. The total credit that a model earns in field f of document d is therefore:

$$I_{d,f} = \sum_{(p,g) \in M_{d,f}} s\,(p, g) \qquad (3)$$

where $M_{d,f}$ is the set of matched pairs per field f of document d.

[13] For example, a Colleague entry of {"FirstMention_Year": "1998", "Name": ["A", "B"]} is split into two items: (A, 1998) and (B, 1998).

Micro Lexical F1, Precision, and Recall: The primary metric of this project is the micro lexical F1 score (Micro LexF1), along with its two components – lexical precision (LexP) and lexical recall (LexR). LexP divides the total credit by the number of items that the model predicted (n^P), while LexR divides the same credit by the number of items in the gold-standard JSONs (n^G). The term "micro" means that credits and item counts are pooled across all fields and all documents *before* the division:

$$\text{LexP} = \frac{\sum_{d,f} I_{d,f}}{\sum_{d,f} n^P_{d,f}}, \qquad \text{LexR} = \frac{\sum_{d,f} I_{d,f}}{\sum_{d,f} n^G_{d,f}} \tag{4}$$

$$\text{Micro LexF1} = \frac{2(\text{LexP})(\text{LexR})}{\text{LexP} + \text{LexR}} = \frac{2\sum_{d,f} I_{d,f}}{\sum_{d,f} n^P_{d,f} + \sum_{d,f} n^G_{d,f}} \tag{5}$$

where $n^P_{d,f}$ and $n^G_{d,f}$ are the numbers of predicted and gold items in field f of document d. Like the conventional F1 score, Micro LexF1 is the harmonic mean of precision and recall. However, rather than counting a matched pair of predicted and gold items as entirely correct or entirely wrong (as the conventional F1 does), each matched pair earns *partial credit* equal to its similarity (between 0 and 1). Unpaired items earn no credit but still count in the denominators. Thus, extracting extra items lowers precision, while missing items lowers recall.

Intuitively, Lex-Precision asks: if we pick a random item *from the model's output*, how correct is it on average? Lex-Recall reverses the question: if we pick a random item *from the human-annotated record*, how much of it did the LLM recover? Furthermore, due to having same numerator, the two metrics' ratio ($LexP/LexR$) equals the total number of predicted items relative to gold items, revealing whether a model extracts too much or too little information

Hallucination and Omission Rates: Finally, I report two error rates – hallucination rate & omission rate – that capture predicted items that do not resemble *any* human-annotated item (hallucination) and human-annotated items that no predicted item resembles (omission), for every field f of JSON document d. In other words, they capture entries that are entirely invented or entirely missed. Since an item only counts as hallucination or omission if it has very low similarity (similarity < 0.15) with human-coded items, both rates are conservative estimates of such errors.

$$h(p) = \mathbf{1}\left[\max_{g \in G_{d,f}} s(p,g) < 0.15\right]; \quad o(g) = \mathbf{1}\left[\max_{p \in P_{d,f}} s(p,g) < 0.15\right] \tag{6}$$

$$\text{Hallucination Rate} = \frac{\sum_{d,f} \sum_{p \in P_{d,f}} h\,(p)}{\sum_{d,f} n^P_{d,f}} \tag{7}$$

$$\text{Omission Rate} = \frac{\sum_{d,f} \sum_{g \in G_{d,f}} o\,(g)}{\sum_{d,f} n^G_{d,f}} \tag{8}$$

where $P_{d,f}$ and $G_{d,f}$ are the sets of predicted and gold items in field f of document d.

Evaluation Results

Overall, the results in table 6 suggest that fine-tuning a moderate-size LLM pays off: The finetuned Mistral Small 3.2 (24B) achieves the best scores on most metrics. All five LLM setups rarely fail to produce valid JSONs, with validity rates ranging from 96.5 to 100 percent.

Table 6. Overall Metrics of the Three LLM Setups

	GPT 5.6 Luna (Medium)	GPT 5.6 Sol (Medium)	Gemini 3.7 Flash (Medium)	NuExtract 2.0 (8B)	Mistral S. 3.2 (24B)
Valid JSONs (%)	**100**	*98.824*	*98.824*	96.471	*98.824*
Avg. Jaccard Similarity	0.821	0.834	*0.849*	0.812	**0.862**
Micro LexF1	0.735	0.746	*0.781*	0.749	**0.796**
Micro Lex-Precision	0.775	0.774	**0.823**	0.789	*0.807*
Micro Lex-Recall	0.699	0.719	*0.743*	0.714	**0.786**
Macro LexF1	0.780	0.816	*0.818*	0.783	**0.828**
Micro CosF1 (embeddings)	0.755	0.759	*0.798*	0.773	**0.817**
Hallucination Rate	**0.041**	0.044	**0.041**	0.038	0.046
Omission Rate[14]	0.093	0.089	*0.086*	0.105	**0.067**

In terms of content, the models' extractions are fairly close to the human-annotated records. Their average BoW Jaccard scores range from 0.812 (NuExtract) to 0.862 (Mistral), meaning that most of the unique words in the gold-standard JSONs also appear in the models' outputs. However, all five models score notably lower on Micro LexF1 (from 0.735 to 0.796), suggesting that *identifying and extracting the right content is easier than placing such content into the right entries and subfields*. Once again, the finetuned Mistral takes the lead (0.796), followed by Gemini 3.7 Flash (0.781). Interestingly, the much smaller NuExtract (8B) achieves a Micro LexF1 (0.749) on par with – and even slightly higher than – both GPT 5.6 models (0.746 for Sol and 0.735 for Luna).

A closer look at precision and recall reveals where Mistral's advantage comes from. All five setups have higher precision than recall, which means that they tend to extract *fewer* items than human coders. Particularly, the ratio between Lex-Recall and Lex-Precision (see equation 4) indicates that the GPT models, Gemini, and NuExtract each extract about 90 to 93 percent as many items as the gold-standard records contain: what they extract tends to be accurate, but they leave more information behind. Mistral, on the other hand, extracts about 97 percent as many items as the human coders did, achieving the highest Lex-Recall (0.786) while keeping its Lex-Precision

[14] The invalid JSON extractions (and all fields and subfields within them) are treated as omissions, given their similarity scores (to the corresponding human-annotated JSONs) being zero. This increases the omission rate as well.

(0.807) close to the top. This balance is important for elite network construction, since every missed job or colleague is a potential tie that goes missing.

The two robustness checks largely confirm these patterns. Mistral achieves the highest Macro LexF1 (0.828), which weighs every field equally. Notably, GPT 5.6-Sol's Macro LexF1 (0.816) is almost as high as Gemini's (0.818), despite its much lower Micro LexF1, suggesting that Sol handles many of the smaller fields well but struggles with the high-volume Job field. Additionally, the ranking of the five setups under Micro CosF1 is exactly the same as under Micro LexF1, meaning that the models' relative performances are not simply driven by differences in wording (e.g., "Chief" versus "Head").

Regarding errors, hallucinations are uncommon across the board: the models invent 3.8 to 4.46 percent of the total items within the human-coded JSON records. However, omission rates are higher – which is expected given that the models are prompted and/or trained to be skip uncertain details rather than extract it (see appendix 3 for more details).

With only 83 scorable biographies, some of these differences may reflect sampling variation rather than true differences in performance. To account for this, I compare every pair of models using paired document bootstrap: I resample the test documents with replacement 2,000 times and, in each round, recalculate every model's score on the *same* set of resampled documents.[15] Since some biographies are harder than others for all models and their scores tend to fluctuate together, comparing the models on the same documents removes this shared variation.

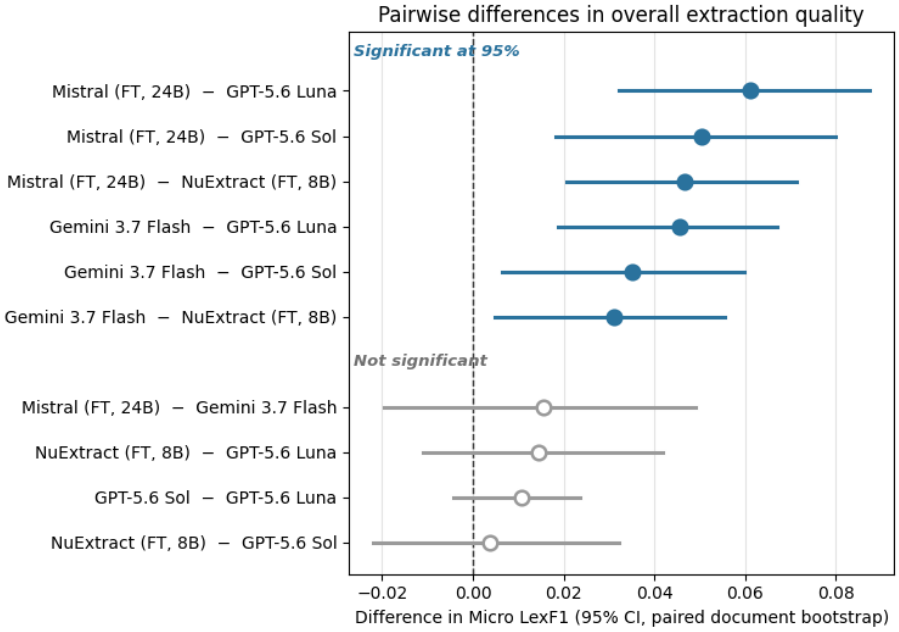


Figure 5. Pairwise Difference in Overall Extraction Quality of the Models

[15] The bootstrapped estimates only include the 15 fields used in the macro averages (see appendix 7), so they can differ slightly (in the third decimal) from the values in table 5.

Overall, the results in figure 5 reveal two distinct tiers of performance. The top tier consists of the fine-tuned Mistral and Gemini 3.7 Flash, while the bottom tier includes the two GPT 5.6 models and the fine-tuned NuExtract. Models within the same tier are statistically indistinguishable, but the distinctions between tiers are clear. On average, Mistral outperforms GPT 5.6-Luna by 0.061 in Micro LexF1 (95% CI: 0.032 to 0.087), GPT 5.6-Sol by 0.051 (0.018 to 0.080), and NuExtract by 0.047 (0.021 to 0.071) – gaps of around 5 to 6 percentage points that are non-trivial when accumulated over thousands of extracted jobs, schools, and relationships. In other words, a moderate-size open LLM, fine-tuned on only 445 examples, can perform just as well as a frontier commercial model on a complex extraction task.

Aside from the overall metrics, figures 6A and 6B show the field-level BoW Jaccard and Lexical F1 scores, respectively.[16] In both figures, the fields are grouped into the four sections of the codebook (table 1), with the number of gold-standard items in each field shown in parentheses.[17] For the basic and non-nested fields like Name, BirthYear, DeathYear, and Ethnicity, all five LLM setups tend to perform strongly, with both BoW Jaccard (figure 6A) and Lexical F1 (figure 6B) mostly between 0.85 and 1.0. The clear exception is Name_Other, where the two GPT models lag far behind (Lexical F1 of 0.52 for Luna and 0.60 for Sol, compared to 0.77 to 0.82 for the other three). Since Gemini (which is not finetuned) also does well in this field, the GPT models' low scores are perhaps due to differences in how they interpret the codebook's rules on which name variants (of the elite in the biography) to record, rather than an inability to find these names.

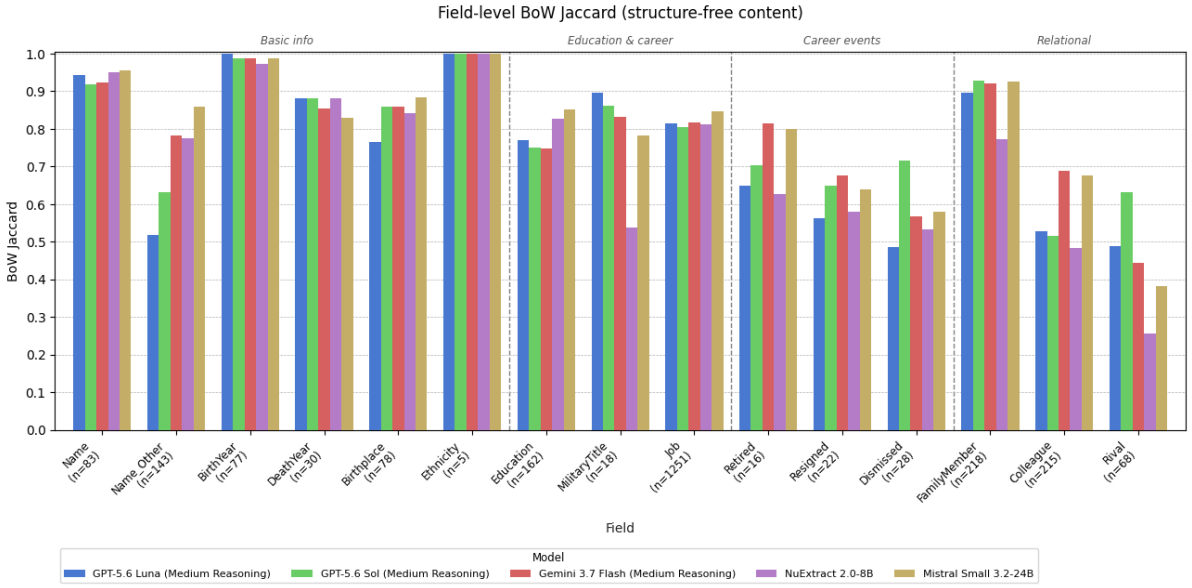


Figure 6A. Field-Level Bag-of-Word Jaccard Across Model Configurations

[16] Scores for fields with few gold-standard items (e.g., Ethnicity with 5 items or MilitaryTitle with 18) should be interpreted with caution, since a single error can change these scores substantially.

[17] Educational and professional details are the most structurally complex and lengthy, with each JSON record having up to dozens of long Education and Job items/entries. There are 1251 Job items in the entire test set.

On the other hand, both the BoW Jaccard and lexical F1 also show that fields containing multiple entries of nested information (e.g., Education, or Job) are notably more challenging. For these two fields, the models' BoW Jaccard scores (0.75 to 0.85) are consistently higher than their Lexical F1 scores (0.70 to 0.81), suggesting that the models find most of the correct content but sometimes place it in the wrong entry or subfield. This is especially true for the Job field. With up to dozens of long and nested entries per elite, Job is where the models are most likely to put organization and sub-organization to the wrong slots, or normalize the position titles differently from human coders. Mistral performs best in both Job and Education, with LexF1 scores of 0.78 and 0.81, respectively – and given that Job alone makes up about half of all gold-standard items, this advantage likely explains much of Mistral's lead in Micro LexF1. In contrast, Gemini performs relatively poorly in Education (0.71), while GPT 5.6-Sol has the lowest Job score (0.70).

Career events (Retired, Resigned, and Dismissed) are rare in the test set (16 to 28 items each) and can be hard to distinguish from each other – for example, a resignation under political pressure may look a lot like a dismissal. My codebook also requires the models to copy the supporting sentences from the biographies into the Detail subfield (figure 4) to justify its decisions, and such long sequences are difficult for generative LLMs to reproduce exactly (especially for generative models with all-decoder architectures). Still, Gemini and Mistral can extract retirements quite well (with Lexical F1 of 0.90 and 0.89, respectively), while resignations and dismissals remain more challenging for all models (0.63 to 0.81). Interestingly, GPT 5.6-Sol performs best at extracting dismissals (0.81).

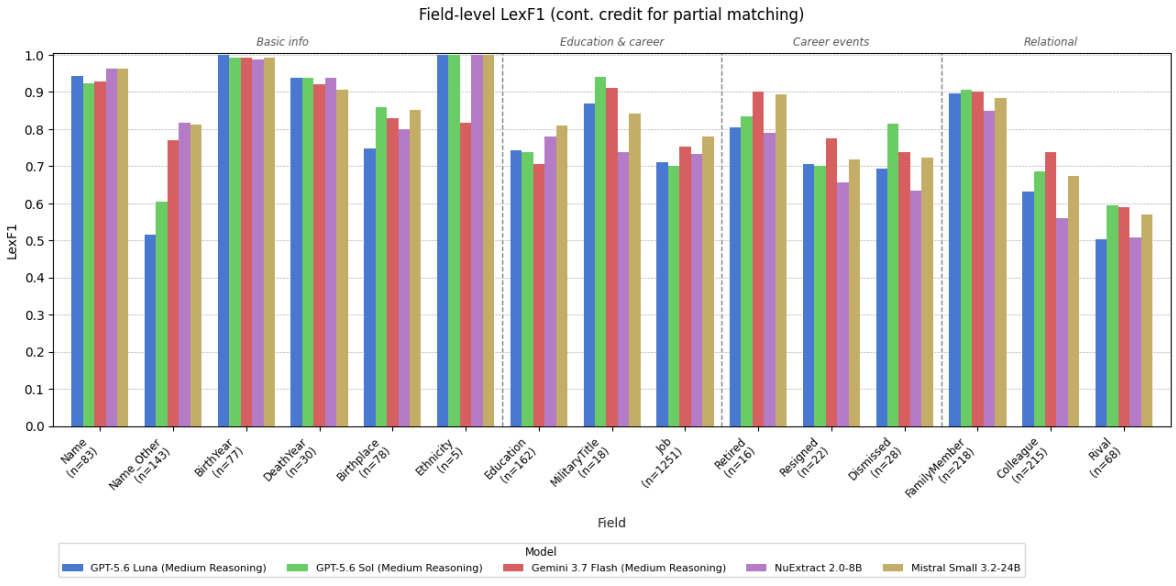


Figure 6B. Field-Level Lexical F1 Across Model Configurations

Similarly, relational fields like Colleague and Rival also proved difficult to extract correctly (Lexical F1 of 0.56 to 0.74 for Colleague and 0.50 to 0.59 for Rival). However, the reasons are perhaps different from the career-event fields. First, the models have to reason about which types of information in the biographical text denote explicit instances of professional and/or

political collaboration or rivalry. For example, it can be easy for an LLM to misunderstand critics of an elite as rivals. Second, the task is further complicated when the models are required to detect the year that such a relation is first mentioned in the text. Third, the models also have to decide whether and when each entry of colleagues or rivals contains a single or multiple persons. This is particularly challenging for elites who not only have long careers but also have taken part in notable events, resulting in various social interactions throughout a lengthy biography.

In these relational fields, Gemini performs best for Colleague (0.74), while Sol and Gemini perform best for Rival (both around 0.59). Notably, the finetuned models tend to over-extract relations. Mistral records 232 colleague items (compared to 215 in the gold-standard records), roughly one in five of which has no counterpart in the human-annotated data (appendix 3). Since hallucinated colleagues and rivals can create false ties in an elite network, this pattern calls for extra human review of relational fields, especially when using the finetuned models.

The models' stronger performance on family relationships (around 0.85 to 0.93) lends some support to this explanation. While FamilyMember also contains nested entries, each entry refers to a single named person, and family ties tend to be described in particular parts of a biography (usually the beginning or the end) – leaving the models with fewer inferential decisions to make.

Together, these results suggest that the finetuned Mistral performs especially well on the high-volume fields that matter most for network construction (Job and Education), while fields that require logical inference remain challenging for all models. Notably, no single model performs best in every field – a point I return to in the conclusion.

Network Example & Surface Validity

To further demonstrate the potential and surface validity of this approach, I include a qualitative example of a network extracted by the fine-tuned Mistral extractor (with some human correction). Figure 7 shows the network of elites in Vietnam's government cabinet in 1999, constructed from the fine-tuned Mistral Small 32B's extracted JSON records. The JSON outputs were cleaned by human correction and then used to construct the network. The ties are based on (1) co-working relations (using the Job field) (detected with the rules described in section 2), (2) explicitly mentioned colleagues (using the Colleague field – with human verification), and (3) educational peers that went to the same school, at the same program of studies, and graduated within 2 years of one another. Node sizes reflect eigenvector centrality, and colors represent sub-groups (or factions) detected by the Leiden community detection algorithm.

Overall, the network reflects historical records of Vietnam at the time. Particularly, while Vietnam's government still overwhelmingly included politicians who were veteran fighters and combatants during the Vietnam War, they are not at all a monolithic group. The community/subgroup in purple comprises mostly of renown military generals, who served as North Vietnam's troops during the Vietnam War and then went on to become high-level officials during the Vietnamese-Khmer War in the 1980s. This pattern is consistent with the military's longstanding

yet limited influence in Vietnamese politics (reflected by most generals' relative separation from the rest of the elites).

The green faction comprises of elites who served in the South, yet many of them worked in high-level economic policy roles, including Vo Van Kiet and Phan Van Khai (ex-prime ministers who also previously served in many economic decision-making bodies), or Tran Xuan Gia (head of Vietnam's central bank).

The other factions comprised of (in general) younger bureaucrats who had primarily worked in civilian and specialized government agencies. Specifically, the community in blue contains mostly younger technocrats who spent years in both local and central government roles across various fields, such as Nguyen Thi Binh (famous diplomat-turned-educator), Le Huy Ngo (agriculture), Nong Duc Manh (forestry and engineering), Le Minh Huong (intelligence), and Nguyen Khoa Diem (communications and propaganda). Two other, albeit more smaller and more disconnected groups, comprise of Do Muoi (the former leader) along with other technocrats such as Nguyen Cong Tan (agriculture) and Nguyen Manh Kiem (engineering and construction). Nearby this community, Nguyen Thi Hang (labor economics and policies).

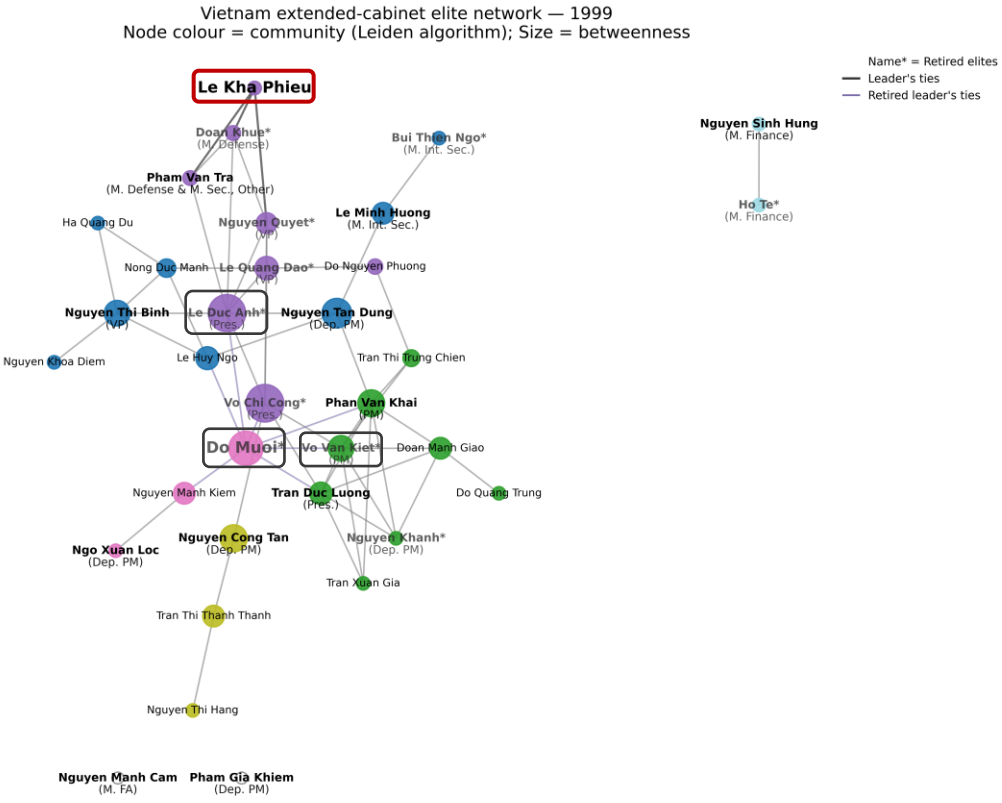


Figure 7. Vietnam's Cabinet Elite Network (1999)

The individual elites' positions in the network also match the power dynamics of Vietnam's elite politics then. In 1999, Vietnam's top leader was Lieutenant General Le Kha Phieu (highlighted in red), who was serving as General Secretary of the Communist Party of Vietnam (CPV). Despite

being two years into his tenure, Phieu had a relatively low centrality and sat at the periphery of the network. In contrast, the three retired elites (in black boxes) – Gen. Le Duc Anh (former president), Vo Van Kiet (former prime minister), and Do Muoi (former CPV general secretary) – had notably high centrality. The most connected elite in this group – Le Duc Anh – was the commander of Vietnam's military during its war against Cambodia and later spent years in civilian government roles, which explains his connections in both the military and government, despite being grouped into the military faction.

Le Kha Phieu's position vis-à-vis the three old guards explained why Phieu remained heavily constrained during his tenure. To escape from these constraints, Phieu attempted to bring in elites from his home province into the central government as well as to increase his control over military and security apparatus – going so far as to create a special task force to secretly spy on high-ranking elites, including the retired elites (Huy Duc 2012b, 331-333; Ngọc Lễ 2019). This obviously did not bode well with the old guards, who coordinate elites at both the national and provincial levels to Phieu as General Secretary in 2001 (Huy Duc 2012b, 355-359; Ngọc Lễ 2019).

Discussion & Conclusion

Overall, this paper addresses a persistent problem in the study of political elites – and, more broadly, in the many areas of political science that rely on turning long, unstructured texts into structured data. Particularly, research on elite networks is constrained by a circular scarcity problem: the fine-grained data needed to construct elite networks are scattered across long biographies and costly to code by hand, and yet the computational tools that could automate such coding require the very same training data that rarely exist for tasks this complex. This paper tackles this challenge by fine-tuning two moderate-size open LLMs on a few hundred human-annotated and synthetic examples, and by evaluating them against three frontier commercial models with metrics that capture both the content and the placement of the extracted information.

The findings suggest that the circular scarcity problem is not insurmountable. Although the task is indeed difficult even for frontier models (no model reaches an overall Micro LexF1 of 0.80), a 24B model finetuned on fewer than 1000 examples can rival to – and in some comparisons even better than – frontier commercial models, even though the evaluation setup may favor the commercial models (which have reasoning capabilities and receive a longer prompt and nine full examples). This finding matters because much of the evidence on the power of finetuning are related to tasks with relatively short or non-nested outputs and has thousands of training examples (Z. Hu et al. 2023; Zhao et al. 2024).[18] A researcher facing the circular scarcity problem cant still create a similar codebook-friendly system with a modest investment in human annotation, supplemented by synthetic examples.

[18]For example, Zhao et al. (2024) fine-tune each of their 310 models for 40,000 training steps on established NLP benchmark datasets.

Moreover, the finetuned Mistral is the best-performing model in extracting the type of high-volume fields that matter most for network construction – Job, Education, and FamilyMember – with few missing or invented entries. Its remaining errors in these fields mostly come from entries that are only partially correct – such as a wrong standardization of the name of an organization, department, or position title – which are relatively quick for human reviewers to fix (at least compared to coding an entire biography from scratch).

Limitations and Future Directions

Despite the positive signs, the models and framework in this paper are not without limitations. First, the evaluation compares the models' outputs against human-annotated records, which are not perfect. As a result, facts correctly extracted by a model but missed by human coders are counted as errors. Second, this paper does not quantify the contribution of synthetic data. While my experience shows that synthetic examples both increase model performance and adaptability to unseen cases, future work should examine how much each component of the training data (human-annotated biographies versus synthetic examples) contributes to the models' performance.

Third, career events (especially resignations and dismissals) and relational fields (especially rivals) remain difficult for every model. These fields are rare in the training data and require the models to judge whether a textual signal denotes a particular event or relationship (e.g., whether a critic is a rival, or whether a forced departure is a resignation or a dismissal). Extracting these fields reliably will likely require a larger amount of (and more targeted) training data – along with human review. In the meantime, *extracted information on career events and relational ties (except for family ties) can only be used for analyses after very careful human verification*.

Relatedly, no single model performs best in every field: GPT 5.6-Sol rarely invents colleagues or rivals but omits many of them, whereas the fine-tuned Mistral misses the fewest colleagues but hallucinates the most. Future research should therefore explore ways to combine extractors – directing human attention to where they disagree – to improve data quality.

Finally, like all human- and machine-coded data, the errors made by this LLM-based framework will also be carried into downstream analyses. While the conservative tendency of the models may reduce false positive extractions and false connections in a network, it also increases the likelihood of false negatives. A partially incorrect or a fully omitted entry can break a tie that would otherwise be formed. One important direction is to examine how the extraction errors documented here propagate into network measures and influence substantive conclusions, which would help researchers decide how much human review is enough for a given research question.

Beyond elite networks, the approach in this paper can be adapted to many other political science problems that require the extraction of actors (individuals and groups), their attributes, their relationships, and events from long texts. For example, it can detect the actions of political actors during disasters or crises, trace careers of legislators and business executives, and unveil interactions among parties, firms, or civil society organizations. For such tasks, the recipe here can

be applied with relatively modest resources: translating a codebook into a nested output template, hand-annotating a few hundred documents, supplementing them with synthetic examples for rare categories, fine-tuning an open model, and evaluating the outputs.

The evaluation framework may also be useful. Conventional classification metrics are not suited to outputs whose entries can be partially correct, listed in different orders, or split and merged differently from the gold standard. An evaluation toolset that includes item-level matching with partial credit, separate measures of content coverage and placement accuracy, and error types offers a clearer picture of what a model gets right and wrong.

All in all, this paper shows that the combination of a carefully designed codebook, a small but information-rich training set, and parameter-efficient fine-tuning can turn a moderate-size open LLM into a capable (though imperfect) coder of complex, nested political information. By integrating such models with human expertise, political scientists can lower the cost of building fine-grained datasets that were previously too resource-intensive to collect.

References

Baturo, Alexander, and Johan A. Elkink. 2016. “Dynamics of Regime Personalization and Patron–Client Networks in Russia, 1999–2014.” *Post-Soviet Affairs* 32(1): 75–98. doi:10.1080/1060586X.2015.1032532.

Benoit, Kenneth, Scott De Marchi, Conor Laver, Michael Laver, and Jinshuai Ma. 2025. “Using Large Language Models to Analyze Political Texts through Natural Language Understanding.” *American Journal of Political Science* 00(n/a): 1–17. doi:10.1111/ajps.70050.

Brandt, Patrick T., Sultan Alsarra, Vito D’Orazio, Dagmar Heintze, Latifur Khan, Shreyas Meher, Javier Osorio, and Marcus Sianan. 2026. “Extractive versus Generative Language Models for Political Conflict Text Classification.” *Political Analysis* 34(3): 344–72. doi:10.1017/pan.2025.10027.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. “Language Models Are Few-Shot Learners.” doi:10.48550/arXiv.2005.14165.

Burnham, Michael, Kayla Kahn, Ryan Yang Wang, and Rachel X. Peng. 2026. “Political DEBATE: Efficient Zero-Shot and Few-Shot Classifiers for Political Text.” *Political Analysis* 34(3): 329–43. doi:10.1017/pan.2025.10028.

Cheung, Jerry, Yuchen Zhuang, Yinghao Li, Pranav Shetty, Wantian Zhao, Sanjeev Grampurohit, Rampi Ramprasad, and Chao Zhang. 2024. “POLYIE: A Dataset of Information Extraction from Polymer Material Scientific Literature.” In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, eds. Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, 2370–85. doi:10.18653/v1/2024.naacl-long.131.

Constantin, Alexandre, Liam Cripwell, Nathan Fradet, Soren Dreano, and Etienne Bernard. 2025. “NuExtract 2.0: Outclassing Frontier LLMs in Information Extraction - NuMind.” *NuExtract 2.0: Outclassing Frontier LLMs in Information Extraction*. https://www.numind.ai/blog/outclassing-frontier-llms----nuextract-2-0-takes-the-lead-in-information-extraction (March 30, 2026).

Coppedge, Michael, John Gerring, Carl Henrik Knutsen, Staffan I. Lindberg, Jan Teorell, Kyle Marquardt, Juraj Medzihorsky, et al. 2026. *V-Dem Methodology V16*. Varieties of Democracy (V-Dem) Project.

Davies, Shawn, Therése Pettersson, and Magnus Öberg. 2026. “Organized Violence 1989–2025, and Violent Political Protests.” *Journal of Peace Research*: xjag046. doi:10.1093/jopres/xjag046.

Erhard, Lukas, Sara Hanke, Uwe Remer, Agnieszka Falenska, and Raphael Heiko Heiberger. 2025. “PopBERT. Detecting Populism and Its Host Ideologies in the German Bundestag.” *Political Analysis* 33(1): 1–17. doi:10.1017/pan.2024.12.

Gerring, John, Erzen Oncel, Kevin Morrison, and Daniel Pemstein. 2019. “Who Rules the World? A Portrait of the Global Leadership Class.” *Perspectives on Politics* 17(4): 1079–97. doi:10.1017/S1537592719000744.

Gleditsch, Nils Petter, Peter Wallensteen, Mikael Eriksson, Margareta Sollenberg, and Håvard Strand. 2002. “Armed Conflict 1946-2001: A New Dataset.” *Journal of Peace Research* 39(5): 615–37. doi:10.1177/0022343302039005007.

Han, Zeyu, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. “Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey.” doi:10.48550/arXiv.2403.14608.

Harbom, Lotta, Erik Melander, and Peter Wallensteen. 2008. “Dyadic Dimensions of Armed Conflict, 1946—2007.” *Journal of Peace Research* 45(5): 697–710. doi:10.1177/0022343308094331.

Hensel, Paul R. 2001. “Contentious Issues and World Politics: The Management of Territorial Claims in the Americas, 1816–1992.” *International Studies Quarterly* 45(1): 81–109. doi:10.1111/0020-8833.00183.

Hensel, Paul R, and Sara McLaughlin Mitchell. 2015. “Lessons from the Issue Correlates of War (ICOW) Project.” *Journal of Peace Research* 52(1): 116–19. doi:10.1177/0022343314546177.

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. “LoRA: Low-Rank Adaptation of Large Language Models.” doi:10.48550/arXiv.2106.09685.

Hu, Yibo, MohammadSaleh Hosseini, Erick Skorupa Parolin, Javier Osorio, Latifur Khan, Patrick Brandt, and Vito D’Orazio. 2022. “ConfliBERT: A Pre-Trained Language Model for Political Conflict and Violence.” In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, eds. Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz. Seattle, United States: Association for Computational Linguistics, 5469–82. doi:10.18653/v1/2022.naacl-main.400.

Hu, Zhiqiang, Lei Wang, Yihuai Lan, Wanyu Xu, Ee-Peng Lim, Lidong Bing, Xing Xu, Soujanya Poria, and Roy Lee. 2023. “LLM-Adapters: An Adapter Family for Parameter-Efficient

Fine-Tuning of Large Language Models." In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, eds. Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, 5254–76. doi:10.18653/v1/2023.emnlp-main.319.

Huguet Cabot, Pere-Lluís, and Roberto Navigli. 2021. "REBEL: Relation Extraction By End-to-End Language Generation." In *Findings of the Association for Computational Linguistics: EMNLP 2021*, eds. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih. Punta Cana, Dominican Republic: Association for Computational Linguistics, 2370–81. doi:10.18653/v1/2021.findings-emnlp.204.

Huy Duc. 2012. *Ben Thang Cuoc II: Quyen Binh*. OSINBOOK.

Ishiyama, John, and Taekbin Kim. 2020. "Authoritarian Survival Strategies and Elite Churn: The Case of North Korea." *International Area Studies Review* 23(2): 160–76. doi:10.1177/2233865920920740.

Josifoski, Martin, Marija Sakota, Maxime Peyrard, and Robert West. 2023. "Exploiting Asymmetry for Synthetic Training Data Generation: SynthIE and the Case of Information Extraction." In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, eds. Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, 1555–74. doi:10.18653/v1/2023.emnlp-main.96.

Keller, Franziska Barbara. 2016. "Moving Beyond Factions: Using Social Network Analysis to Uncover Patronage Networks Among Chinese Elites." *Journal of East Asian Studies* 16(1): 17–41. doi:10.1017/jea.2015.3.

Kim, Taekbin, and John Ishiyama. 2023. "Kim Jong Un and Consolidating Power: Explaining Elite Appointment in the Workers' Party of Korea Central Committee, 2012–2019." *Journal of Asian and African Studies*: 1–14. doi:10.1177/00219096231197771.

King, Gary, and Will Lowe. 2003. "An Automated Information Extraction Tool for International Conflict Data with Performance as Good as Human Coders: A Rare Events Evaluation Design." *International Organization* 57(3): 617–42. doi:10.1017/S0020818303573064.

Kirkpatrick, James, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, et al. 2017. "Overcoming Catastrophic Forgetting in Neural Networks." *Proceedings of the National Academy of Sciences* 114(13): 3521–26. doi:10.1073/pnas.1611835114.

Kuhn, H. W. 1955. "The Hungarian Method for the Assignment Problem." *Naval Research Logistics Quarterly* 2(1–2): 83–97. doi:10.1002/nav.3800020109.

Lampinen, Andrew K., Arslan Chaudhry, Stephanie C. Y. Chan, Cody Wild, Diane Wan, Alex Ku, Jörg Bornschein, et al. 2025. “On the Generalization of Language Models from In-Context Learning and Finetuning: A Controlled Study.” doi:10.48550/arXiv.2505.00661.

Laurer, Moritz, Wouter van Atteveldt, Andreu Casas, and Kasper Welbers. 2024. “Less Annotating, More Classifying: Addressing the Data Scarcity Issue of Supervised Machine Learning with Deep Transfer Learning and BERT-NLI.” *Political Analysis* 32(1): 84–100. doi:10.1017/pan.2023.20.

Li, Yinghao, Rampi Ramprasad, and Chao Zhang. 2024. “A Simple but Effective Approach to Improve Structured Language Model Output for Information Extraction.” In *Findings of the Association for Computational Linguistics: EMNLP 2024*, eds. Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, 5133–48. doi:10.18653/v1/2024.findings-emnlp.295.

Liu, Sheng, Haotian Ye, Lei Xing, and James Zou. 2024. “In-Context Vectors: Making In Context Learning More Effective and Controllable Through Latent Space Steering.” doi:10.48550/arXiv.2311.06668.

Liu, Yu, Duantengchuan Li, Kaili Wang, Zhuoran Xiong, Fobo Shi, Jian Wang, Bing Li, and Bo Hang. 2024. “Are LLMs Good at Structured Outputs? A Benchmark for Evaluating Structured Output Capabilities in LLMs.” *Information Processing & Management* 61(5): 103809. doi:10.1016/j.ipm.2024.103809.

Luo, Yun, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2025. “An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-Tuning.” doi:10.48550/arXiv.2308.08747.

Matthews, Austin S. 2023. “The Politburos of Communist Eastern EuropeIntroducing New Individual-Level Data on Candidate and Full Members.” *Communist and Post-Communist Studies* 56(2): 159–76.

Meyer, David, Victor Shih, and Jonghyuk Lee. 2016. “Factions of Different Stripes: Gauging the Logics of Factions in the Reform Period.” *Journal of East Asian Studies* 16(1): 43–60. doi:10.1017/jea.2015.5.

Mistral AI. 2025. “Mistral Small 3.” https://mistral.ai/news/mistral-small-3 (March 31, 2026).

Ngọc Lễ. 2019. “Lê Đức Anh: làm tướng giỏi, làm chính trị tồi? [Le Duc Anh: good General, bad politician?].” *Voice of America (VoA) Vietnamese*. https://www.voatiengviet.com/a/lê-đức-anh-làm-tướng-giỏi-làm-chính-trị-tồi-/4886896.html (July 10, 2024).

Nyrup, Jacob, and Stuart Bramwell. 2020. “Who Governs? A New Global Dataset on Members of Cabinets.” *American Political Science Review* 114(4): 1366–74. doi:10.1017/S0003055420000490.

Nyrup, Jacob, Carl Henrik Knutsen, Peter Egge Langsæther, and Ina L. Kristiansen. 2024. “Paths to Power: A New Dataset on the Social Profile of Governments.” doi:10.2139/ssrn.4631225.

Oskooei, Amirkia Rafiei, amirkia.oskooei@std.yildiz.edu.tr, https://orcid.org/0009-0004-3490-550X, View Profile, Kaan Baturalp Cosdan, baturalp.cosdan@std.yildiz.edu.tr, https://orcid.org/0009-0000-5216-7003, et al. 2026. “When Many-Shot Prompting Fails: An Empirical Study of LLM Code Translation.” In *Proceedings of the 1st Workshop on Code Translation, Transformation, and Modernization*, ACM Conferences, , 38–43. doi:10.1145/3786180.3788314.

Salehyan, Idean. 2015. “Best Practices in the Collection of Conflict Data.” *Journal of Peace Research* 52(1): 105–9. doi:10.1177/0022343314551563.

Shih, Victor, Christopher Adolph, and Mingxing Liu. 2012. “Getting Ahead in the Communist Party: Explaining the Advancement of Central Committee Members in China.” *American Political Science Review* 106(1): 166–87. doi:10.1017/S0003055411000566.

Stepanov, Ihor, and Mykhailo Shtopko. 2024. “GLiNER Multi-Task: Generalist Lightweight Model for Various Information Extraction Tasks.” http://arxiv.org/abs/2406.12925 (August 11, 2024).

Stuhler, Oscar, Cat Dang Ton, and Etienne Ollion. 2025. “From Codebooks to Promptbooks: Extracting Information from Text with Generative Large Language Models.” *Sociological Methods & Research* 54(3): 794–848. doi:10.1177/00491241251336794.

Tang, Yongjian, Doruk Tuncel, Christian Koerner, and Thomas Runkler. 2025. “The Few-Shot Dilemma: Over-Prompting Large Language Models.” In *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, , 134–41. doi:10.1109/FLLM67465.2025.11391015.

Tjong Kim Sang, Erik F., and Fien De Meulder. 2003. “Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition.” In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, , 142–47. https://aclanthology.org/W03-0419/ (March 30, 2026).

Trad, Fouad, and Ali Chehab. 2025. “Retrieval-Augmented Few-Shot Prompting Versus Fine-Tuning for Code Vulnerability Detection.” doi:10.48550/arXiv.2512.04106.

Wu, Haolun, Ye Yuan, Liana Mikaelyan, Alexander Meulemans, Xue Liu, James Hensman, and Bhaskar Mitra. 2024. “Learning to Extract Structured Entities Using Language Models.” In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, eds. Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, 6817–34. doi:10.18653/v1/2024.emnlp-main.388.

Xu, Derong, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, et al. 2024. “Large Language Models for Generative Information Extraction: A Survey.” *Frontiers of Computer Science* 18(6): 186357. doi:10.1007/s11704-024-40555-y.

Xu, Lingling, Haoran Xie, Si-Zhao Joe Qin, Xiaohui Tao, and Fu Lee Wang. 2023. “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment.” doi:10.48550/arXiv.2312.12148.

Ye, Junjie, Nuo Xu, Yikun Wang, Jie Zhou, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. “LLM-DA: Data Augmentation via Large Language Models for Few-Shot Named Entity Recognition.” doi:10.48550/arXiv.2402.14568.

Zaratiana, Urchade, Nadi Tomeh, Pierre Holat, and Thierry Charnois. 2024. “GLiNER: Generalist Model for Named Entity Recognition Using Bidirectional Transformer.” In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, eds. Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, 5364–76. doi:10.18653/v1/2024.naacl-long.300.

Zhang, Zikang, Wangjie You, Tianci Wu, Xinrui Wang, Juntao Li, and Min Zhang. 2025. “A Survey of Generative Information Extraction.” In *Proceedings of the 31st International Conference on Computational Linguistics*, eds. Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert. Abu Dhabi, UAE: Association for Computational Linguistics, 4840–70. https://aclanthology.org/2025.coling-main.324/ (August 28, 2025).

Zhao, Justin, Timothy Wang, Wael Abid, Geoffrey Angus, Arnav Garg, Jeffery Kinnison, Alex Sherstinsky, et al. 2024. “LoRA Land: 310 Fine-Tuned LLMs That Rival GPT-4, A Technical Report.” doi:10.48550/arXiv.2405.00732.

Appendices

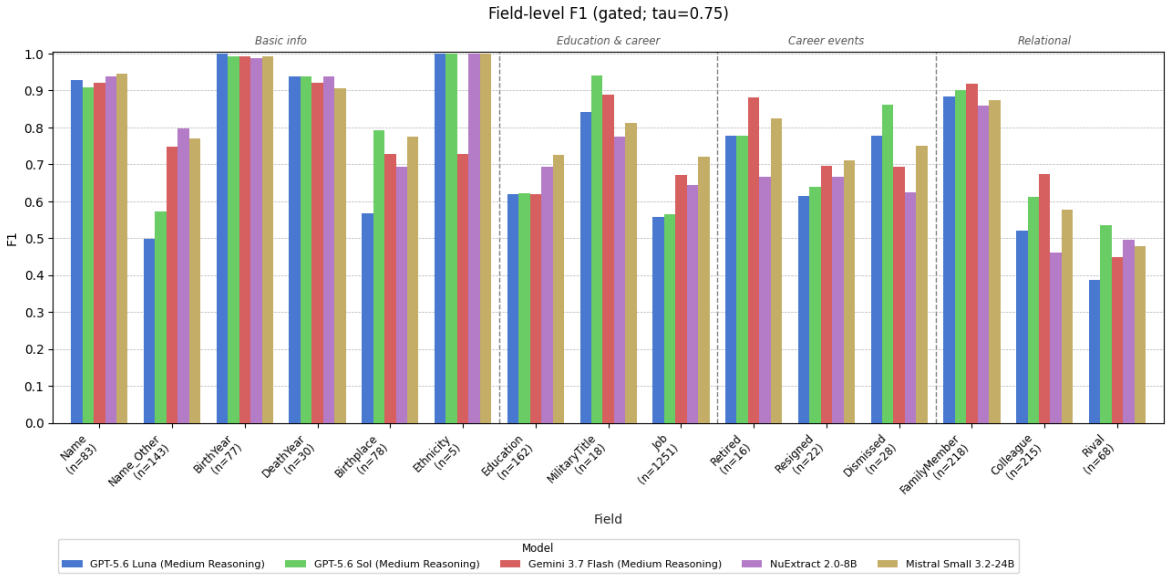
Appendix 1: Field-Level F1 (Strict Version; Matching Threshold: Jaccard = 0.75)

For comparison, appendices 1 and 2 report a stricter, conventional F1 score, in which a matched pair counts as a correct extraction (a true positive, or TP) only if its similarity reaches a threshold τ (0.75 in the main analysis):

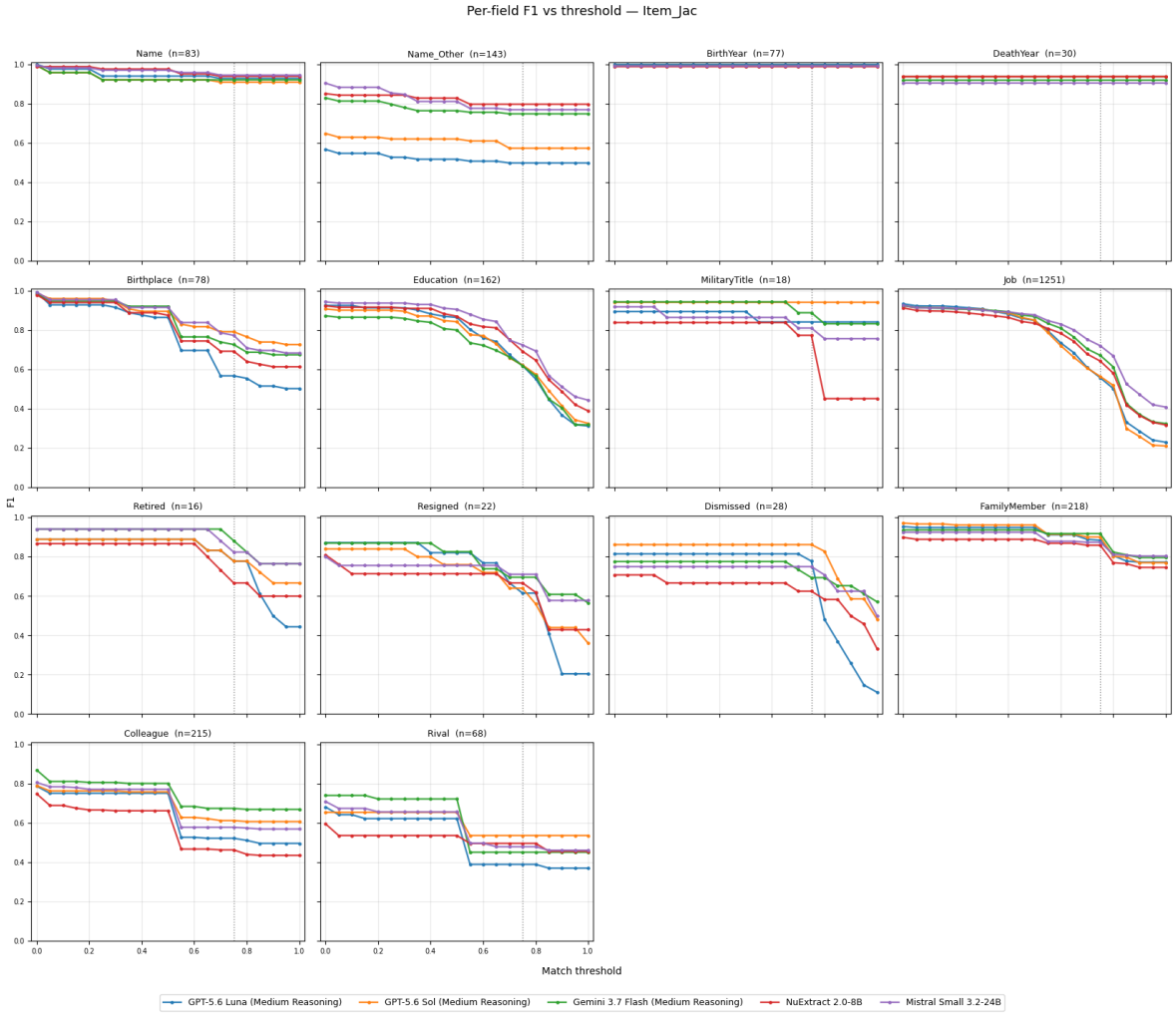
$$TP = |\{(p, g) \in M : s(p, g) \geq \tau\}|, \qquad \text{Strict F1}_\tau = \frac{2\,TP}{\sum_{d,f} n^P_{d,f} + \sum_{d,f} n^G_{d,f}} \qquad (A1)$$

where M pools the matched pairs across all fields and documents. Since every predicted item that is not a true positive is a false positive (FP), and every gold item that is not a true positive is a false negative (FN), equation A1 is equivalent to the familiar formula $2TP/(2TP + FP + FN)$. Comparing equations 5 and A1 reveals the intuition behind Micro LexF1: it is essentially an F1 score in which the *count* of correct items is replaced by the *total credit* earned. I use Micro LexF1 as the primary metric because the strict version depends on an arbitrary threshold, where a single spelling difference can push a nearly correct item below the cutoff (and thus count it as entirely wrong!).

In my use case, the matching threshold tau is $\tau = 0.75$. In other words, the strict F1 version considers the machine-coded version to be a correct detail if its content has a 0.75 similarity score with the human coded version. However, if the predicted item is 0.74 similar to the human coded version, that counts as an incorrect extraction.



Appendix 2: Field-Level F1 (Strict F1) by Matching Threshold



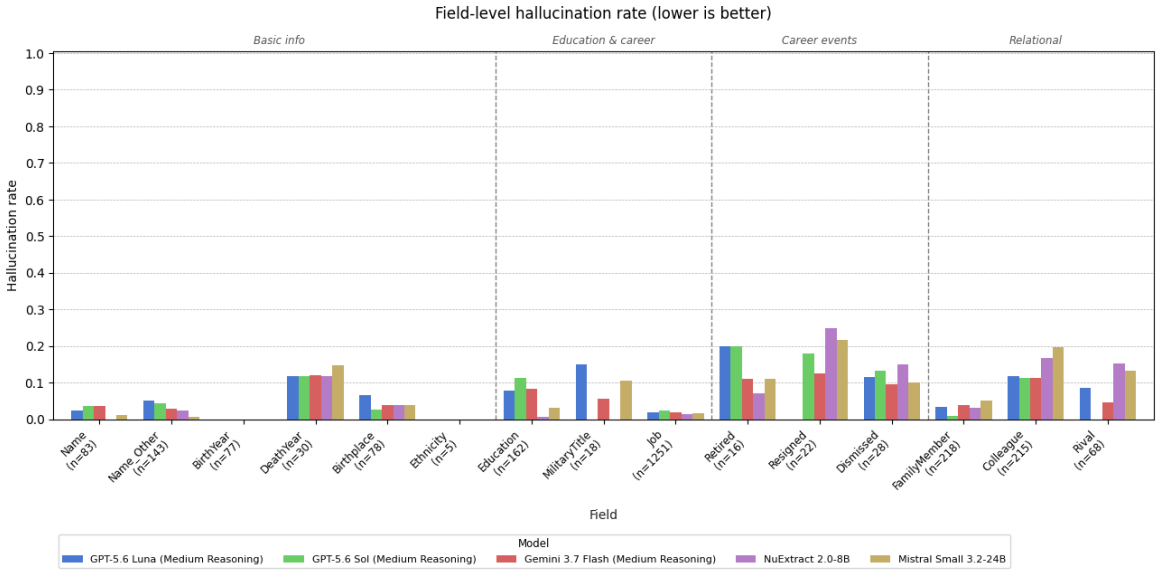
Appendix 3: Hallucination & Omissions Rates by Field

Apart from overall accuracy, it is also useful to examine what kinds of errors the models make in each field. Appendix 3 presents the field-level hallucination and omission rates, which show whether a model tends to miss information entirely or to record information that has no basis in the human-annotated records (recall that both rates only count items with little or no resemblance to anything on the other side).

The two rates mirror precision and recall. Hallucination rate tells us how often the model records something that the manual annotations do not, while omission rate shows how often the model does not extract a piece of information in the human-coded annotations. A completely wrong value in a single-item field (e.g., an incorrect birth year) counts as both a hallucination and an omission. These two types of errors have different consequences for elite network construction. An omitted job or colleague removes some true ties from an elite's network, while a hallucinated organization can create false ties between the elite and everyone else associated with that organization.

Two caveats are also worth noting. First, both rates are conservative: an item escapes these labels as long as it shares a small amount of content with some item on the other side (e.g., a duplicated job does not count as hallucinated), although such errors are still penalized by Lex-Precision and Lex-Recall. Second, hallucination is judged against the human-annotated JSON rather than the biography itself – if the human coders missed a fact that the model correctly extracted, that fact is still counted as a hallucination.

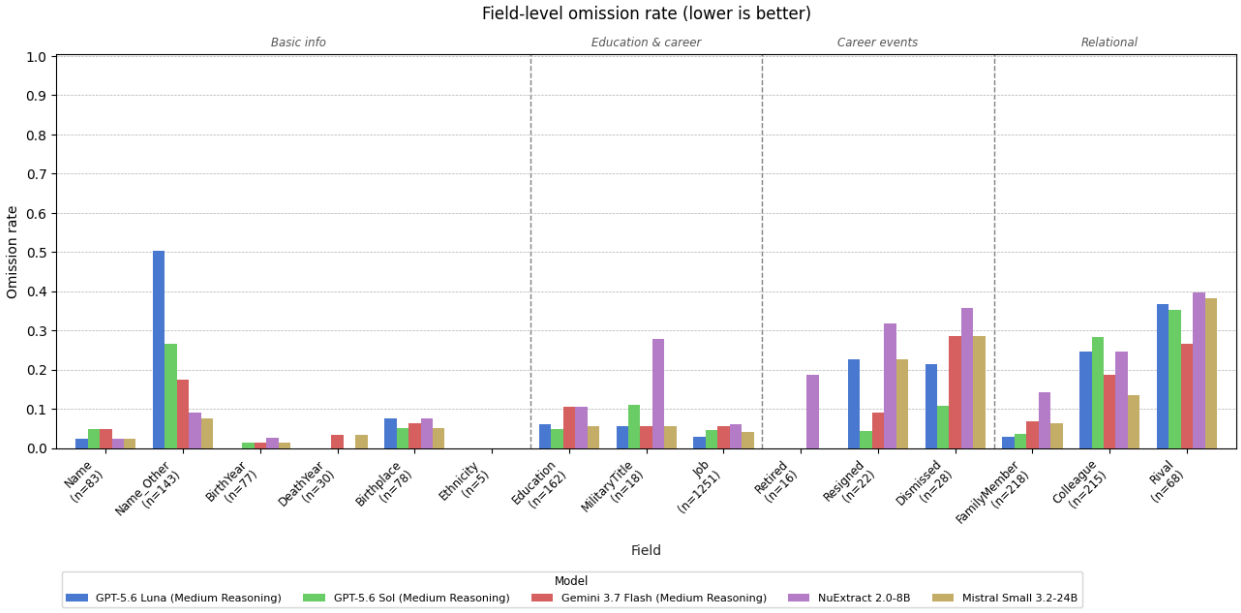
Overall, both hallucinations and omissions are relatively uncommon for the basic and non-nested fields. First, for the high-volume fields that form the backbone of elite network construction – Job, Education, and FamilyMember – both types of errors are relatively rare. For example, the models omit 3 to 6 percent of the 1,251 gold-standard jobs and invent/hallucinate about 2 percent of their predicted jobs. In other words, the models rarely miss or make up an entire job. Their imperfect Lexical F1 scores in these fields therefore mostly come from entries that are only partially correct (e.g., missing details or dates attached to the wrong position), rather than entries that are entirely missing or invented.



Second, some low field-level scores are driven by one type of error in particular. The GPT models' weak performance in Name_Other, for instance, comes almost entirely from omissions: GPT 5.6-Luna misses about half of all alternative names (and GPT 5.6-Sol about a quarter), even though all models rarely record aliases that do not exist. In fact, this single field accounts for about a third of all items that Luna misses.

Third, errors are much more common in rare fields that require more inference, namely career events and relationships. Resignations and dismissals are frequently missed, and the finetuned models tend to record resignations that do not exist in the gold-standard records – perhaps because a forced departure can be read as either type of event. Rival is the most difficult field overall: every model misses more than a quarter of the gold-standard rivals.

Relational fields also reveal different trade-offs across models. GPT 5.6-Sol rarely invents colleagues or rivals but misses many of them, while the finetuned Mistral misses the fewest colleagues but invents the most. Gemini strikes a better balance between the two.


Field-level omission rate (lower is better)
Basic info
Education & career
Career events
Relational
Omission rate
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Name (n=83)
Name_Other (n=143)
BirthYear (n=77)
DeathYear (n=30)
Birthplace (n=78)
Ethnicity (n=5)
Education (n=162)
MilitaryTitle (n=18)
Job (n=1251)
Retired (n=16)
Resigned (n=22)
Dismissed (n=28)
FamilyMember (n=218)
Colleague (n=215)
Rival (n=68)
Field
Model
GPT-5.6 Luna (Medium Reasoning)
GPT-5.6 Sol (Medium Reasoning)
Gemini 3.7 Flash (Medium Reasoning)
NuExtract 2.0-8B
Mistral Small 3.2-24B

Appendix 4: How the Evaluation Metrics Respond to Common Extraction Errors

Distinctions Between Lexical Metrics

Although Micro LexF1 and BoW Jaccard both reward similar words with human-coded records, Micro LexF1 also rewards putting them in the correct JSON fields and keys. First, correct words placed in the wrong entry or subfield still earn credit under BoW Jaccard but not Micro LexF1, which compares items one by one. Second, BoW Jaccard averages across documents, while Micro LexF1 pools all items across the test set, giving elites with longer careers greater weight. Third, Micro LexF1 penalizes extra and missing items through its denominators, which BoW Jaccard overlooks.

Figure 5 demonstrates these differences with a hypothetical example of the human-coded and predicted Job field in an elite's JSON record. Reach record contains three job entries/items that are almost exactly the same. The only difference lies in the **Org_Main_OtherName** key of the second Job entry/item: the gold JSON contains "CPV" (the Communist Party of Vietnam's abbreviated name), whereas the predicted JSON leaves this key empty.

Gold JSON:

```
  "Job": [
  {
    "Start": "1995", "End": "1998", "Position": "Director", "Location": [],
    "Org_Main_Name": "Ministry of Foreign Affairs", "Org_Main_OtherName": "",
    "Org_Department": "Bureau for Northeast Asia", "Org_Unit": ""
  },
  {
    "Start": "September 1998", "End": "December 1999", "Position": "Head", "Location": ["Vietnam"],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "CPV",
    "Org_Department": "Central International Relations Committee", "Org_Unit": ""
  },
  {
    "Start": "January 2000", "End": "2002", "Position": "Deputy Minister", "Location": [],
    "Org_Main_Name": "Ministry of Foreign Affairs", "Org_Main_OtherName": "",
    "Org_Department": "", "Org_Unit": ""
  },
],
```

Predicted JSON:

```
  "Job": [
  {
    "Start": "1995", "End": "1998", "Position": "Director", "Location": [],
    "Org_Main_Name": "Ministry of Foreign Affairs", "Org_Main_OtherName": "",
    "Org_Department": "Bureau for Northeast Asia", "Org_Unit": ""
  },
  {
    "Start": "September 1998", "End": "December 1999", "Position": "Head", "Location": ["Vietnam"],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "",
    "Org_Department": "Central International Relations Committee", "Org_Unit": ""
  },
  {
    "Start": "January 2000", "End": "2002", "Position": "Deputy Minister", "Location": [],
    "Org_Main_Name": "Ministry of Foreign Affairs", "Org_Main_OtherName": "",
    "Org_Department": "", "Org_Unit": ""
  },
],
```

Figure 5. Hypothetical example of field-level (Job) between gold-standard and predicted JSON.

Since only one of the 28 unique words in this field ("CPV") is missing, the BoW Jaccard for this field is 0.964. Under Micro LexF1, the first and third entries are perfect matches, while the second entry of the predicted and human-coded JSONs has six out seven entries matching perfectly – giving it a similarity of $6/7 \approx 0.857$. With three predicted and three gold items, the Micro LexF1

for this field is $2 \times (1 + 0.857 + 1) / (3 + 3) \approx 0.952$. Of course, in reality, the gap between the two metrics can be much larger, especially when models misplace information across entries or duplicate them (appendix 4).

Prediction (vs. gold Job field in figure 5)	**BoW Jaccard**	**Lex-P**	**Lex-R**	**LexF1**	**Strict F1 ($\tau = 0.75$)**	**Halluc. Rate**	**Omit. Rate**
"CPV" missing in Other_Name (figure 5)	0.964	0.952	0.952	0.952	1.000	0.00	0.00
Third job left out entirely	0.821	1.000	0.667	0.800	0.800	0.00	0.00
First job listed twice	1.000	0.750	1.000	0.857	0.857	0.00	0.00
Location ("Vietnam") attached to the wrong job	1.000	0.897	0.897	0.897	1.000	0.00	0.00
One invented job (no overlap with any gold job)	0.875	0.750	1.000	0.857	0.857	0.25	0.00
One invented job sharing a start year with a gold job	0.903	0.750	1.000	0.857	0.857	0.00	0.00
Invalid JSON output	0.000	0.000	0.000	0.000	0.000	–	1.00

Notably, BoW Jaccard does not change at all when a job is duplicated or a location is attached to the wrong job, while LexF1 penalizes both. Similarly, the hallucination and omission rates remain at zero in several cases where the model clearly adds or drops a job, since the added (or dropped) job still shares some content with another entry. These errors are captured instead by Lex-Precision and Lex-Recall.

Appendix 5: Micro CosF1

Micro Cosine F1 (Embeddings): Lexical metrics have an obvious weakness: they only reward identical words. A model that writes "Premier" instead of "Prime Minister," or "Chief" instead of "Head," receives no credit for that subfield even though the meaning is (nearly) the same. As a robustness check, I therefore report the micro cosine F1 (Micro CosF1), which follows the same logic as Micro LexF1 but replaces the word-based similarity in equation 2 with a meaning-based (semantic) similarity. Specifically, each subfield value is converted into a sentence embedding – a numerical vector representing the meaning of a text – using the all-mpnet-base-v2 model (Reimers and Gurevych 2019).[19] The similarity between two values is based on the cosine of the angle between their embedding vectors, which is then rescaled:

$$s^{Emb}(p,g) = \frac{1}{|K_{p,g}|} \sum_{k \in K_{p,g}} \max\left\{0,\ \frac{\cos\big(e(p_k), e(g_k)\big) - b}{1-b}\right\} \qquad (A5a)$$

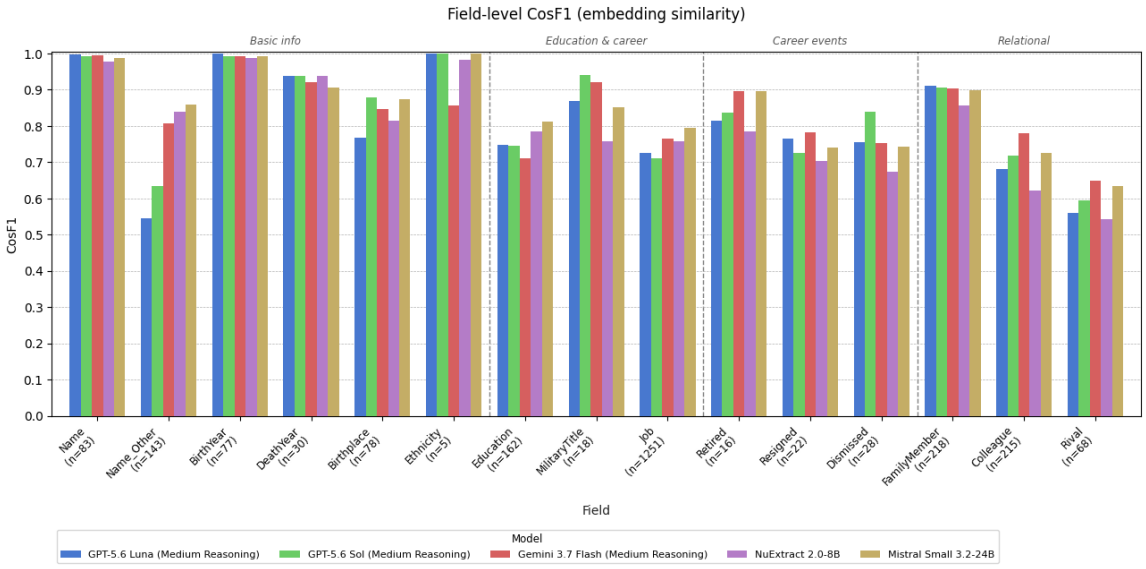
$$\text{Micro CosF1} = \frac{2\sum_{d,f} I_{d,f}^{Emb}}{\sum_{d,f} n_{d,f}^{P} + \sum_{d,f} n_{d,f}^{G}} \qquad (A5b)$$

where $e(\cdot)$ is the embedding vector of a subfield value and $I_{d,f}^{Emb}$ is the total credit after re-pairing the items (with the Hungarian algorithm) using s^{Emb}. The rescaling step is necessary because embedding models rarely give completely unrelated texts a cosine similarity near zero. In this test set, two randomly chosen (and different) gold-standard values of the same subfield – for instance, two unrelated job positions – have an average cosine similarity of $b = 0.4485$. Subtracting this baseline and stretching the result back to the 0–1 range means that unrelated values score around 0, while identical values still score 1 – a common practice in embedding-based evaluation (Zhang et al. 2020). The baseline is calculated once from the gold-standard data and then held fixed, so that all models are scored on the same scale.

Intuitively, Micro CosF1 is a more forgiving grader that accepts paraphrases. However, the risk here is that the embeddings-based CosF1 can also treat "1995" and "1996," or two different people sharing the same family name, as quite similar – even though they actually are not – thereby overestimating the models' extraction accuracy. For this reason, Micro CosF1 serves as a secondary robustness check.

[19] Like other sentence-embedding models, all-mpnet-base-v2 places texts with similar meanings close to each other in a high-dimensional space. A cosine similarity of 1 means that two vectors point in exactly the same direction (i.e., essentially identical meaning), while values closer to 0 indicate unrelated texts.

Appendix 6: Field-Level Micro CosF1



Appendix 7: Macro LexF1 Explanation

Macro Lexical F1, Precision, and Recall: While Micro LexF1 tells us how accurate the models are across all extracted information, it can hide weak performances in fields that contain fewer items but are nonetheless substantively important – such as career events or rivals. To address this, I also report the macro lexical F1 (Macro LexF1). Instead of pooling all items together, I first calculate the lexical precision, recall, and F1 separately for each field f (pooling across documents):

$$\text{LexP}_f = \frac{\sum_d I_{d,f}}{\sum_d n^P_{d,f}}, \qquad \text{LexR}_f = \frac{\sum_d I_{d,f}}{\sum_d n^G_{d,f}} \qquad (A7a)$$

$$\text{LexF1}_f = \frac{2(\text{LexP}_f)(\text{LexR}_f)}{\text{LexP}_f + \text{LexR}_f} = \frac{2\sum_d I_{d,f}}{\sum_d \left(n^P_{d,f} + n^G_{d,f}\right)} \qquad (A7b)$$

and then take the simple average across fields:

$$\text{Macro LexF1} = \frac{1}{|F^*|} \sum_{f \in F^*} \text{LexF1}_f \qquad (A7c)$$

where F^* is the set of 15 fields with at least five gold-standard items in the test set.[20] Macro Lex-Precision and Macro Lex-Recall are calculated in the same way, by averaging LexP_f and LexR_f

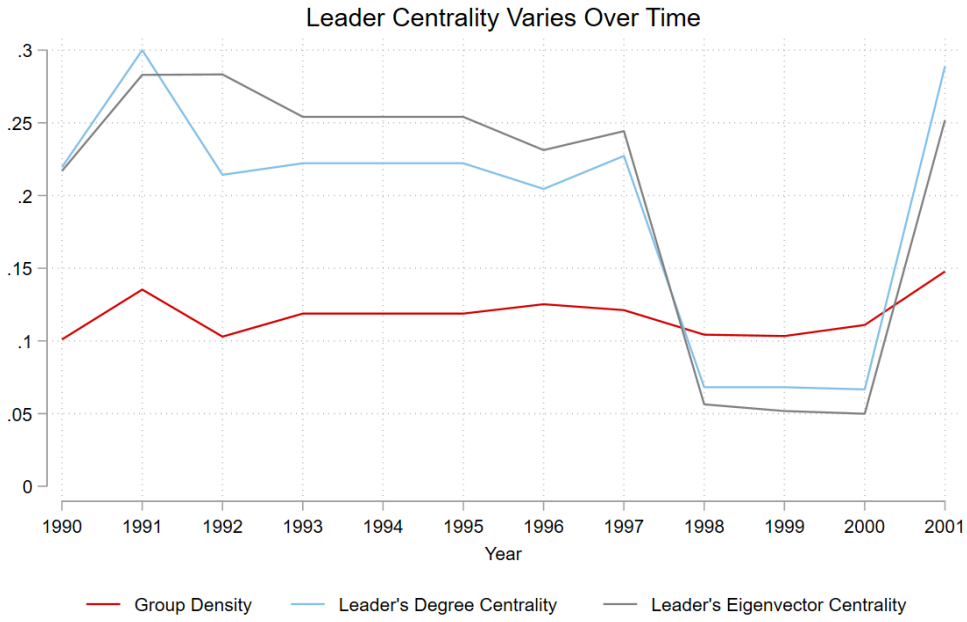
[20] The three remaining fields in the template (Arrested, Exiled, and Killed) each have fewer than five gold-standard items in the test set. These fields are still included in all micro metrics (so that, for example, an invented arrest still

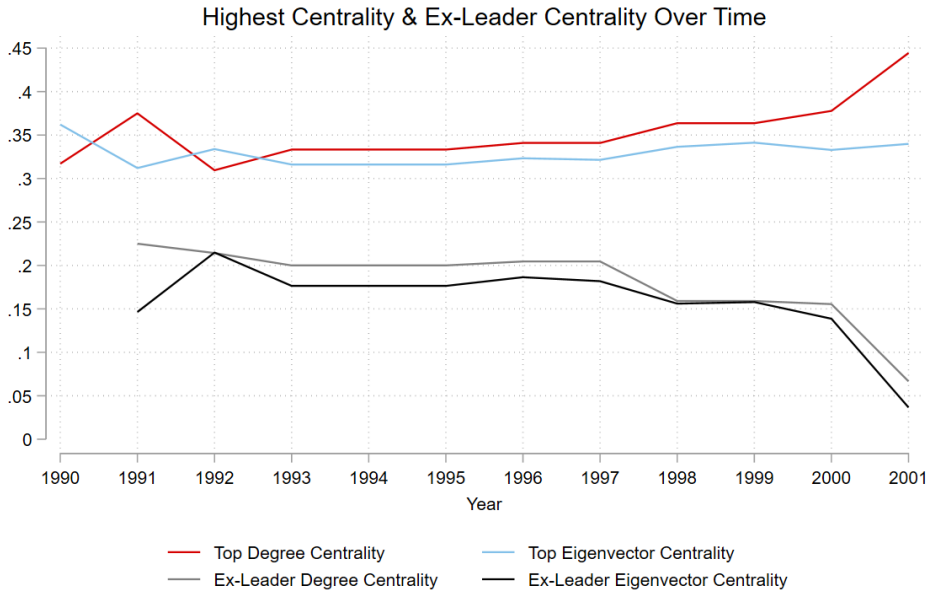
across the fields in F^*. The field-level LexF1 scores in figure 6B are exactly the LexF1_f values in equation 8. Two technical details are worth noting. First, if a model extracts nothing for a field that has gold items, that field's precision is counted as 0 rather than dropped. Second, Macro LexF1 is the average of field-level F1 scores – not the harmonic mean of Macro Lex-Precision and Macro Lex-Recall.

The difference between the two F1 scores therefore lies in their weighting. Micro LexF1 weights every *item* equally, so it mostly reflects a model's performance on high-volume fields like Job, FamilyMember, and Colleague. On the other hand, Macro LexF1 weights every *field* equally, so the Ethnicity field (5 items) counts as much as the Job field (1,251 items). Put differently, Micro LexF1 answers the question "how accurate is the dataset that the model produces, piece by piece?", while Macro LexF1 answers "how well does the model handle a typical field in the codebook?". A model whose Macro LexF1 is much higher than its Micro LexF1 is likely doing relatively well on the many smaller fields but struggling with the high-volume ones. Because some fields rest on only a handful of items, Macro LexF1 is also noisier than its micro counterpart.

lowers a model's Micro Lex-Precision) but are left out of the macro averages. Otherwise, a field that appears in only one or two documents would move the macro average as much as the Job field, which contains 1,251 items.

Appendix 8: Variations in Some Network Measures Over Time in Vietnam





Appendix 9: Example JSON Output & Codebook

Example of a real annotated JSON from an elite's biography.


```
{
"Name": "Nguyen Phu Trong",
"BirthYear": "1944",
"DeathYear": "2024",
"Birthplace": {"City": "Hanoi", "City_OtherName": [], "Region": "Hanoi", "Region_OtherName": []},
"Education": [
   {"School": "Nguyen Gia Thieu High School", "OtherName": [],
    "Start": "1957", "End": "1963", "Level": "High School", "Field": "", "Location": ["Gia Lam","Long Bien","Hanoi"]},
   {"School": "Hanoi University", "OtherName": [],
    "Start": "1963", "End": "", "Level": "University", "Field": "Literature", "Location": ["Hanoi"]},
   {"School": "Nguyen Ai Quoc School", "OtherName": ["Ho Chi Minh National Academy of Politics"],
    "Start": "August 1973", "End": "April 1976", "Level": "", "Field": "Political Economy", "Location": []},
   {"School": "Academy of Social Sciences", "OtherName": ["Academy of Social Sciences of the Central Committee of the Communist Party of the Soviet Union"],
    "Start": "September 1981", "End": "August 1983", "Level": "Doctorate", "Field": "Historical Sciences", "Location": ["USSR"]}
  ],
"Job": [
   {"Start": "December 1967", "End": "September 1981", "Position": "Staff", "Location": [],
    "Org_Main_Name": "Hoc Tap", "Org_Main_OtherName": "Communist Magazine", "Org_Department": "", "Org_Unit": ""},
   {"Start": "August 1983", "End": "October 1983", "Position": "Staff", "Location": [],
    "Org_Main_Name": "Communist Magazine", "Org_Main_OtherName": "Communist Review", "Org_Department": "Party Building Department", "Org_Unit": ""},
   {"Start": "October 1983", "End": "September 1987", "Position": "Deputy Head", "Location": [],
    "Org_Main_Name": "Communist Magazine", "Org_Main_OtherName": "Communist Review", "Org_Department": "Party Building Department", "Org_Unit": ""},
   {"Start": "September 1987", "End": "March 1989", "Position": "Head", "Location": [],
    "Org_Main_Name": "Communist Magazine", "Org_Main_OtherName": "Communist Review", "Org_Department": "Party Building Department", "Org_Unit": ""},
   {"Start": "March 1989", "End": "May 1990", "Position": "Editorial Board Member", "Location": [],
    "Org_Main_Name": "Communist Magazine", "Org_Main_OtherName": "Communist Review", "Org_Department": "", "Org_Unit": ""},
   {"Start": "May 1990", "End": "August 1991", "Position": "Deputy Editor-in-Chief", "Location": [],
    "Org_Main_Name": "Communist Magazine", "Org_Main_OtherName": "Communist Review", "Org_Department": "", "Org_Unit": ""},
   {"Start": "August 1991", "End": "August 1996", "Position": "Editor-in-Chief", "Location": [],
    "Org_Main_Name": "Communist Magazine", "Org_Main_OtherName": "Communist Review", "Org_Department": "", "Org_Unit": ""},
   {"Start": "January 1994", "End": "", "Position": "Member", "Location": ["Vietnam"],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "CPV", "Org_Department": "Central Committee", "Org_Unit": ""},
   {"Start": "August 1996", "End": "January 2000", "Position": "Deputy Secretary", "Location": ["Hanoi"],
    "Org_Main_Name": "City Party Committee", "Org_Main_OtherName": "Hanoi Party Committee", "Org_Department": "", "Org_Unit": ""},
   {"Start": "August 1996", "End": "January 2000", "Position": "Head", "Location": ["Hanoi"],
    "Org_Main_Name": "City Party Committee", "Org_Main_OtherName": "Hanoi Party Committee", "Org_Department": "University Affairs Committee", "Org_Unit": ""},
   {"Start": "March 1998", "End": "November 2001", "Position": "Vice Chairman", "Location": [],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "CPV", "Org_Department": "Central Theoretical Council", "Org_Unit": ""},
   {"Start": "December 1997", "End": "", "Position": "Member", "Location": ["Vietnam"],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "CPV", "Org_Department": "Politburo", "Org_Unit": ""},
   {"Start": "August 1999", "End": "2000", "Position": "Member", "Location": ["Vietnam"],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "CPV", "Org_Department": "Politburo", "Org_Unit": "Standing Committee"},
   {"Start": "January 2000", "End": "June 2006", "Position": "Secretary", "Location": ["Hanoi"],
    "Org_Main_Name": "City Party Committee", "Org_Main_OtherName": "Hanoi Party Committee", "Org_Department": "", "Org_Unit": ""},
   {"Start": "November 2001", "End": "", "Position": "Chairman", "Location": [],
    "Org_Main_Name": "Central Theoretical Council", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""},
   {"Start": "2002", "End": "2024", "Position": "Member", "Location": [],
    "Org_Main_Name": "National Assembly", "Org_Main_OtherName": "", "Org_Department": "Hanoi Delegation", "Org_Unit": ""},
   {"Start": "June 2006", "End": "July 2011", "Position": "Chairman", "Location": [],
    "Org_Main_Name": "National Assembly", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""},
   {"Start": "January 2011", "End": "July 2024", "Position": "General Secretary", "Location": ["Vietnam"],
    "Org_Main_Name": "Communist Party of Vietnam", "Org_Main_OtherName": "CPV", "Org_Department": "", "Org_Unit": ""},
   {"Start": "January 2011", "End": "July 2024", "Position": "Secretary", "Location": [],
    "Org_Main_Name": "Central Military Commission", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""},
   {"Start": "2013", "End": "2024", "Position": "Head", "Location": [],
    "Org_Main_Name": "Central Steering Committee on Anti-Corruption and Negative Phenomena", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""},
   {"Start": "October 2018", "End": "April 2021", "Position": "President", "Location": ["Vietnam"],
    "Org_Main_Name": "Presidency", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""},
   {"Start": "October 2018", "End": "April 2021", "Position": "Honorary President", "Location": ["Vietnam"],
    "Org_Main_Name": "Vietnam Red Cross Society", "Org_Main_OtherName": "", "Org_Department": "", "Org_Unit": ""},
   {"Start": "", "End": "July 2024", "Position": "Standing Member", "Location": ["Vietnam"],
    "Org_Main_Name": "Public Security", "Org_Main_OtherName": "", "Org_Department": "Central Party Committee", "Org_Unit": ""}
  ],
"FamilyMember": [
   {"Name": "Ngo Thi Man", "Relation": "Wife"},
   {"Name": "Nguyen Thi Kim Ngoc", "Relation": "Daughter"},
   {"Name": "Nguyen Trong Truong", "Relation": "Son"}
  ]
}
```

The codebook below summarizes the coding rules that guided both the human annotators and the models. The rules are adapted from the training prompt (appendix 10B) and presented in a more readable format; examples are drawn from figures 3 and 5 or from the training data.

Field	Subfield(s)	Definition and coding rules	Example
General rules	Evidence	Use only information explicitly stated or directly supported by the biography. Hedged claims ("reportedly," "likely") are extracted if no contradictory evidence exists. Claims described as rumors, rejected, or erroneous are not extracted. Sections that record uncertainty, identity risks, source conflicts, or research gaps are never used as evidence.	"He reportedly served as deputy director (1985–1988)" → extracted.
	Ambiguity and conflicts	Prefer omission over speculation. When two values conflict and the biography does not resolve the conflict, omit the disputed field or entry.	Sources give 1950 and 1951 as the birth year (unresolved) → BirthYear omitted.
	Duplicates	Do not repeat the same person, event, degree, rank, or job unless the biography establishes them as distinct (different dates, terms, or contexts).	Two terms as Minister (1992–1995; 1998–2000) → two Job entries.
	Dates	"Month YYYY" if the month is known; "YYYY" if only the year is known; "Present" for an explicitly ongoing role; empty otherwise. For an event spanning a range, the Date is the start.	"appointed on 12 March 1998" → "March 1998".
	Output	One single-line, valid JSON object following the template; fields with no information are removed; no null values.	---
Name	---	The elite's principal standardized name.	"Viktor Yerin"
Name_Other	--- (list)	Explicit aliases, nicknames, noms de guerre, former/birth/married names, alternate transliterations, initials, and native-script forms. Bare titles, ranks, or honorifics are excluded unless they function as an established alias.	["Viktor Erin"]; ["Олег Дмитриевич Давыдов"]
BirthYear, DeathYear	---	Year of birth and death (if any).	"born on 25 May 1940" → "1940"
Birthplace	City, City_OtherName, Region, Region_OtherName	City: the most specific named inhabited or municipal-level place. Region: the larger first-level administrative or geographic unit. Modern administrative successors are not inferred unless the biography establishes them.	"born in Yekaterina, Sverdlovsk oblast" → City: "Yekaterina", Region: "Sverdlovsk"
Ethnicity	--- (list)	Only when explicitly stated for the elite or her parents. Never inferred from surname, birthplace, citizenship, residence, language, or religion.	---

Field	Subfield(s)	Definition and coding rules	Example
Education	School, OtherName, Start, End, Level, Field, Location	Formal schooling, military/professional academies, postgraduate study, and structured training programs. Employment at a school is a Job, not Education. Field may be broadened to a stable category. Degree levels follow the template's standard values and are not conflated (e.g., a "candidate" degree is not automatically a doctorate).	School: "Moscow Institute of Civil Engineering"; OtherName: ["MISI"]; End: "1963"; Level: "University"; Field: "Engineering"; Location: ["Moscow"]
MilitaryTitle	Title, YearReceived	Formal military (or equivalent) service ranks, with the year received when known.	"promoted to Colonel General on May 9, 1992" → Title: "Colonel General"; YearReceived: "1992"
Job	Position	The most specific formal title stated; no generic label (e.g., "Member") when a more specific title is given. Concurrent positions are separate entries. Formal temporary roles (e.g., Head of Delegation) may count; mere attendance, speeches, or visits do not.	"From 1992, he served as Head of Operations Department, Minister of Internal Affairs of Russia until 1995" → Start: "1992"; End: "1995"; Position: "Minister"; Location: ["Russia"] Org_Main_Name: "Ministry of Internal Affairs"; Location: ["Russia"] Org_Department: "Design Bureau"
	Org_Main_Name, Location	Government bodies: the jurisdiction in the official name is removed and placed first in Location. Non-government organizations (universities, firms, parties): geographic names stay in the name. Historical jurisdictions are preserved as stated. Headquarters locations are not inferred.	
	Org_Main_OtherName	An explicitly stated alternative name (e.g., an abbreviation) of the main organization (single string).	
	Org_Department, Org_Unit	Named subordinate bodies (committee, bureau, department) within the main organization. For military jobs, the service branch is the main organization; named formations (army, corps, division, brigade, regiment, battalion) go in the department/unit fields; military districts, commands, and theaters go in Location (unless both a district and a geographic location are stated).	

Field	Subfield(s)	Definition and coding rules	Example
Career events	Date, Detail	Retired: the end of an entire career (not just one position). Resigned: a voluntary or pressured departure framed as a resignation. Dismissed: forced or administrative removal, relief of duties, expulsion from a governing body, or removal through a purge or coup. Arrested: actual arrest, detention, house arrest, or imprisonment (not mere investigation or indictment). Exiled: forced exile, deportation, or compelled flight (not voluntary emigration). Killed: assassination, execution, or other intentional killing (not natural death, illness, or accident). Detail contains the complete supporting sentences from the biography, as separate list items. One episode can produce several events with the same date.	A coup leads to both removal and detention → one Dismissed and one Arrested entry with the same Date.
FamilyMember	Name, Relation	Named relatives only (not "his wife" without a name). The relative's principal name as stated (gendered surname forms are preserved). The most specific relation label, with the correct direction of the relationship (e.g., in-laws).	Name: "Anna Borisovna Davydova"; Relation: "Wife"
Colleague	FirstMention_Year, Name	An explicit, meaningful professional association: working together, collaboration, mentorship, sponsorship or nomination, or close political/factional alliance. A superior counts only if such a relationship is shown. Mere co-occurrence, holding a similar office, succession, or being quoted about the elite does not count.	"…assisted by Andrei Dunaev, … first deputy since 1992" → {FirstMention_Year: "1992", Name: ["Andrei Dunaev"]}
Rival	FirstMention_Year, Name	An explicit political, professional, or factional struggle: stated rivalry, factional conflict, a struggle for institutional control, a purge, a coup, or an active effort to remove, arrest, exile, or harm the elite. Criticism, ordinary policy disagreement, journalism, prosecutorial or judicial roles, electoral competition, or membership in an opposing party do not count by themselves.	---
Colleague, Rival	(both)	FirstMention_Year is the earliest year in which that specific relationship is stated (empty if undated). The same person can be both a colleague and a rival at different times. Several names are grouped into one entry only if they share the same year and context. Bare titles (e.g., "the Minister") are not names.	---

Appendix 10: Promptbook and Inference Settings

Table A10 summarizes how each model was used.

Table A10. Inference Settings

Setting	Finetuned models (NuExtract 2.0-8B, Mistral Small 3.2-24B)	Commercial models (GPT 5.6-Luna, GPT 5.6-Sol, Gemini 3.7 Flash)
System prompt	Training prompt (appendix 10B), training and inference prompts are identical	Inference prompt (appendix 10C)
User message	JSON template + biography	JSON template + nine annotated examples + biography
Examples in the prompt	None	Nine (four from Vietnam, three from Russia, two from Cambodia), drawn at random (seed 42) from the training split; one per elite, with original (non-paraphrased) versions preferred
Decoding	Greedy (no sampling)	Temperature 0 and seed 42 requested (dropped automatically when an API does not support them); Medium reasoning effort
Output format control	None	JSON mode (response_format = json_object) for GPT & Gemini models
Context and output limits	Context limit: 18000 tokens Output limit: 3000 tokens	Context limit: 25000 tokens Output limit: 6000 tokens

Appendix 10B: Training Prompt (Finetuned Models)

```
You are a structured information extraction system for political elite biographies.

TASK
Extract information from the biography of the elite (subject) into the target JSON schema.

BIOGRAPHY SECTIONS
The biography may be structured like Wikipedia entries. The biography may also contains sections
that are explicitly labeled as uncertainty, identity risk, source conflict, or unresolved
research gaps rather than accepted fact.

OUTPUT REQUIREMENTS
- Return exactly one single-line valid JSON object. No explanation, markdown, or text outside the
JSON.
- Preserve the target schema and its data types.
- Keep nested sub-fields inside a retained object even when some values are empty.
- Remove a root-level field when all its values/sub-fields are empty.
- Do not include a list item when all its sub-fields are empty.
- Do not output null.
- Fields typed as a single string (e.g. an "OtherName" field on an organization) must stay a
string, not a list; if several aliases exist, keep the most useful or most explicitly supported
one.

============================
R0. GENERAL EXTRACTION RULES
============================
0.0 Use only information explicitly stated or directly supported by the biography.
0.1 If evidence is ambiguous, prefer omission over speculation. Do not guess missing dates,
names, organizations, locations, ranks, degrees, or relationships beyond what a rule below
expressly permits. Do not invent people, organizations, or events.
0.2 Silence about how well a claim is corroborated is not ambiguity — extract it normally.
```

0.3 Extract a supported claim even if hedged ("reportedly", "likely", "not yet corroborated") as long as no contradictory evidence exists.
0.4 When two or more values conflict and the biography does not resolve the conflict or mark one as authoritative, omit the disputed field/entry rather than choosing independently.
0.5 Preserve multiple distinct entries when supported, but do not duplicate the same person, event, education record, military title, or job unless the biography establishes them as distinct (different dates, terms, or contexts).
0.6 A notable event mentioned in passing is not automatically a Job — a Job requires an identifiable formal position or role.

==========
R1. IDENTITY FIELDS
==========
Name: the subject's principal standardized name.
Name_Other: explicit aliases, nicknames, noms de guerre, former/birth/married names, alternate transliterations, initials, and native-script forms given alongside a romanized form. Exclude bare titles/ranks/honorifics unless the full expression functions as an established alias.
Ethnicity: extract only when explicitly stated for the subject or the subject's parents. Do not infer ethnicity from surname, birthplace, citizenship, residence, language, or religion.

==========
R2. DATES
==========
Normalize Start/End/event dates to:
1. "Month YYYY" when a specific month and year are known (drop the day).
2. "YYYY" when only the year is known.
3. "Present" for an explicitly ongoing role/status.
Leave a date field empty ("") when it is unknown and no ongoing/ending status is stated — do not guess.
For an event spanning a range, use the starting Month/Year as the event's Date field and preserve the full range in any detail field.

==========
R3. EDUCATION
==========
Extract educational institutions, dates, degree/study level, field, and location when supported, include formal schooling, military/professional academies, postgraduate study, and structured training programs.
Employment at an academic institution is a Job, not Education.
Field of study may be normalized to a broader stable category when clearly appropriate (e.g. a specific engineering discipline → "Engineering").
Use the schema's standard level values as literally as they map from the source language; do not conflate distinct degree levels (e.g. a country-specific "candidate" degree is not automatically the same as a doctorate).

==========
R4. MILITARY TITLES
==========
Extract only formal military or equivalent service ranks, with the year received when known.

==========
R5. BIRTHPLACE NORMALIZATION
==========
Use the most specific named inhabited/municipal-level place as the city/district field, and the larger first-level administrative or geographic unit (province, state, region, or historical equivalent) as the region field.
Do not infer a modern administrative successor unless the biography itself establishes the correspondence.

==========
R6. JOB EXTRACTION
==========
Extract official or professional positions with dates, location, and organization when supported. This includes government, legislative, party, military/security, diplomatic, academic, business, and other clearly defined professional roles.
An organization-name field may be left empty when the biography establishes an independent professional role (e.g. "Writer", "Consultant") without naming an organization.

Use the most specific formal position stated; do not add a generic label like "Member" or "Participant" when a more specific title (Secretary, Chairman, Director, Commander, Minister, Head, etc.) is given in the same passage.
Create separate Job entries for distinct concurrent positions, even if dates and organization overlap.
Preserve historically or institutionally specific titles as stated rather than replacing them with a generic modern equivalent.
Formal temporary or delegated roles — like Head of Delegation, Chair, Negotiator, Signatory, Founder, or an appointed committee member — may be extracted as Jobs when stated. Mere attendance, giving a speech, or an inspection visit, is not a Job unless the biography states a formal role.

ORGANIZATION AND LOCATION NORMALIZATION:
- For government bodies (national/federal or subnational — ministries, governments, legislatures, courts, military/security agencies, regional/municipal administrations, and comparable government bodies):
If the body's official name explicitly includes a jurisdiction (a country, region, city, or comparable administrative unit), remove that jurisdiction from the main organization-name field and place it first in the location field instead.
Add further workplace locations only when explicitly supported by the text — do not infer a headquarters city just because it's typical for that kind of institution.
Use department/unit fields for a named subordinate body (a committee, bureau, or department) within the main organization if explicitly stated.
- For non-government organizations (schools, universities, hospitals, businesses, media organizations, political parties/movements, and state-owned enterprises):
Do NOT strip geographic names out of these organizations' official names — a university or company's location is normally part of its proper name and should stay in the organization-name field. Separately record a supported physical/operating location if the text gives one.
Use department/unit fields for a named subordinate body within the main organization if stated.
- Historical jurisdictions: preserve a historical state, predecessor country, or former administrative name only when it is explicit in the organization's name, title, or role description at the time. Do not silently update a historical jurisdiction name to its modern successor.

MILITARY JOBS:
Use the service branch ([Country Adjective] Army, [Country Adjective] Navy, [Country Adjective] Air Force, [Country Adjective] Armed Forces, etc.) as the main organization;
Place military districts, military zones, military regions in Location, not the Org_Department field - unless when the biography specify both military district/zone and geographical location (if both are explicitly stated, then put military district/zone to Org_Department and location clue in the Location field). Place commands, threaters, and deployment countries in Location, not the Org_Department field.
Use Org_Department/Org_Unit fields for named formations (largest to smallest: army/corps/division/brigade/regiment/battalion) only when explicitly stated.

==========
R7. CAREER-ENDING AND ADVERSE EVENTS
==========
Distinguish by what the biography actually describes, not by assumption:
- Retirement: the end of an entire active career, not merely one position while the person continued working elsewhere.
- Resignation: a voluntary or pressured departure still framed as resignation.
- Dimissmal: Forced/administrative removal, relief of duties, expulsion from a governing body, or removal through a purge/coup.
- Arrest: actual arrest, detention, house arrest, or imprisonment. Do not use this category for a mere investigation, accusation, or indictment. A later release, pardon, or acquittal can be noted as detail without changing the event type.
- Exile: forced exile, deportation, expulsion, or compelled flight under explicit threat - NOT voluntary emigration, ordinary relocation, or foreign employment.
- Killed: assassination, execution, extrajudicial killing, or intentionally inflicted death - NOT natural death, illness, or accident.
When a single episode (e.g. a coup or purge) produced multiple distinct entries date (e.g. both dismissal and arrest) with the same same date.
Detail fields should be complete sentences or full bullet points drawn from the biography, kept as separate list items when the source presents them separately — do not merge multiple distinct sentences into one string.

```
====================
R8. FAMILY MEMBERS
====================
Extract only named family members — do not extract unnamed references (e.g. "his wife", "two
sons") without a name.
Use the relative's stated principal name; full-name completion is allowed only when the identity
is clear and directly supported by context.
Do not mechanically alter a relative's surname to match the subject's — preserve gendered or
otherwise distinct surname forms as stated.
Use the most specific supported relationship label (e.g. Wife, Husband, Son, Daughter, Father,
Mother, Sibling, Grandparent, in-law relations) and reason correctly about relationship direction
(e.g. the parent of a subject's child's spouse is an in-law of the subject, not a direct
relative).


=========================
R9. COLLEAGUE AND RIVAL
=========================
Colleague: extract a person as a colleague when the biography establishes an explicit, meaningful
professional association — working together, collaboration, mentorship, sponsorship/nomination,
close political/factional alliance, or comparable explicit relationship. A superior may count as
a colleague when a meaningful working/mentorship/alliance relationship is shown, not merely
because they formally appointed the subject. Do not extract a colleague relationship merely from
co-occurrence in the same passage, holding a similar office, succession, or a person being quoted
about the subject.


Rival: extract a person as a rival for an explicit political, professional, factional, or power
struggle — stated rivalry, factional conflict, a struggle for institutional control, a purge, a
coup, or an active effort to remove/arrest/exile/harm the subject's political position. Do not
infer rivalry merely from criticism, ordinary policy disagreement, journalism,
prosecutorial/judicial roles, electoral competition, or simply being in an opposing party. A
person who formally executed a routine dismissal/arrest is not automatically a rival unless the
biography indicates they personally organized or drove the adverse action.


Use the earliest year explicitly stated for that specific relationship (not the person's first
appearance in the biography generally); leave the year empty if the relationship is explicit but
undated. The same person may appear as both colleague and rival at different times if the
biography documents a change in the relationship — keep both records rather than deleting the
earlier one.
Group multiple names into one entry only when they share both the same confirmed year and the
same event/context; otherwise create separate entries.
Do not extract bare titles (e.g. "the General", "the Minister") as a person's name when no actual
name is given.
```

Appendix 10C: Inference Prompt (Commercial Models)

```
You are a structured information extraction system for political elite biographies.


TASK
Extract information from the biography of the elite (subject) into the target JSON schema.
NOTE: The biography may be about either a fake or real elite, about real or synthetic events.
Regardless of the factual nature of the biography, extract them ALL as best as you can based on
the rules and instructions below.


BIOGRAPHY SECTIONS
The biography may be structured like Wikipedia entries, or compiled into labeled sections (e.g.
###0-###10). Apply the same rules either way. If any section or passage is explicitly labeled or
framed as uncertainty, identity risk, source conflict, or an unresolved research gap, treat it as
reference only — never extract evidence from it, since it records what is NOT yet accepted fact.


OUTPUT REQUIREMENTS
- Return exactly one single-line valid JSON object. No explanation, markdown fences, or text
outside the JSON.
- Preserve the target schema and its data types.
- Keep nested sub-fields inside a retained object even when some values are empty.
- Remove a root-level field when all its values/sub-fields are empty.
- Do not include a list item when all its sub-fields are empty.
```

- Do not output null.
- Fields typed as a single string (e.g. an "OtherName" field on an organization) must stay a string, not a list; if several aliases exist, keep the most useful or most explicitly supported one.

==========
R0. GENERAL EXTRACTION RULES
==========
0.0 Use only information explicitly stated or directly supported by the biography.
0.1 If evidence is ambiguous, prefer omission over speculation. Do not guess missing dates, names, organizations, locations, ranks, degrees, or relationships beyond what a rule below expressly permits. Do not invent people, organizations, or events.
0.2 Silence about how well a claim is corroborated is not ambiguity — extract it normally.
0.3 Extract a supported claim even if hedged ("reportedly", "likely", "not yet corroborated") as long as no contradictory evidence exists.
0.4 When two or more values conflict and the biography does not resolve the conflict or mark one as authoritative, omit the disputed field/entry rather than choosing independently.
Example: one source gives a birth year of 1953, another implies roughly 1966-1967, and the biography does not resolve the conflict → omit BirthYear entirely.
0.5 Preserve multiple distinct entries when supported, but do not duplicate the same person, event, education record, military title, or job unless the biography establishes them as distinct (different dates, terms, or contexts).
0.6 A notable event mentioned in passing is not automatically a Job — a Job requires an identifiable formal position or role.
Example: "In 1996 he attended a regional security conference" → not a Job unless a formal position or role is stated.

==========
R1. IDENTITY FIELDS
==========
Name: the subject's principal standardized name, in the form/order most consistently used in the biography. Naming conventions vary by country (patronymics, family-name-first order, multi-word surnames); preserve the biography's own presentation rather than reformatting it.
Name_Other: explicit aliases, nicknames, noms de guerre, former/birth/married names, alternate transliterations, initials, and native-script forms given alongside a romanized form. When a native-script form appears mixed into one phrase with a romanized name (e.g. "Name (Native Script)"), extract only the native-script characters as a clean standalone entry, not the mixed phrase. Exclude bare titles/ranks/honorifics unless the full expression functions as an established alias.
Do not extract titles, ranks, honorifics, academic styles, or forms of address (e.g. Mr./Mrs., Dr., Professor, Excellency, Prince/Princess, General, Colonel, or any country-specific royal/religious honorific) unless the full expression functions as an established, stable identifying alias consistently used in the biography. Do not extract a standalone honorific (e.g., "Dr.," a bare royal or religious title) without an attached personal name.
Multi-word or multi-syllable personal names are common in many countries; do not split one person's name across separate entries when parsing Name_Other, Colleague, Rival, or FamilyMember — parse full name boundaries carefully when several people are listed together (commas/semicolons usually separate distinct people).
  Note: these rules also apply when parsing Colleague and Rival names.
Ethnicity: extract only when explicitly stated for the subject or the subject's parents. Do not infer ethnicity from surname, birthplace, citizenship, residence, language, or religion.

==========
R2. DATES
==========
Normalize Start/End/event dates to:
1. "Month YYYY" when a specific month and year are known (drop the Day).
2. "YYYY" when only the year is known.
3. "Present" for an explicitly ongoing role/status.
Leave a date field empty ("") when it is unknown and no ongoing/ending status is stated — do not guess.

Split a combined range at the dash, normalizing each side independently:
* 05/1975–1979 → Start="May 1975", End="1979"
* 1968–1970 → Start="1968", End="1970"
* March–November 1997 → Start="March 1997", End="November 1997"

For an event spanning a range, use the starting Month/Year as the event's Date field and preserve the full range in a detail field.
Example: "1975–1979: imprisoned by the ruling authority." → Date="1975", Detail=["1975–1979: imprisoned by the ruling authority."]

==========
R3. EDUCATION
==========
Extract educational institutions, dates, degree/study level, field, and location when supported; include formal schooling, military/professional academies, postgraduate study, and structured training programs.
Employment at an academic institution is a Job, not Education.
Field of study may be normalized to a broader stable category when clearly appropriate (e.g. a specific engineering discipline → "Engineering").
Standard Level values: Primary, Middle, Secondary, High School, University, Master, Doctorate, Training, Military, Honorary. Use "Training" for short non-degree courses (political/cadre training, seminars, management programs). Do not conflate distinct degree levels — a country-specific "candidate" degree is not automatically the same as a doctorate.

==========
R4. MILITARY TITLES
==========
Extract only formal military or equivalent service ranks, with the year received when known. A command or administrative position (e.g. Commander, Chief of Staff, Minister of Defense) is a Job, not a MilitaryTitle.

Examples: Lieutenant, Captain, Major, Lieutenant Colonel, Colonel, Brigadier General, Major General, Lieutenant General, General, and equivalent ranks in any stated national armed force. Do not extract command or administrative positions (Commander, Commander-in-Chief, Chief of Staff, Chief of Military Operations, Minister of Defence/Military Affairs) as MilitaryTitle — these are Jobs.

==========
R5. BIRTHPLACE NORMALIZATION
==========
Use the most specific named inhabited/municipal-level place as the city/district field, and the larger first-level administrative or geographic unit (province, state, region, or historical equivalent) as the region field.
Example: "Village X, District Y, Province Z" → City="District Y", Region="Province Z" (the village-level detail is dropped unless it functions as an alternate identifying label).
Example: "Village A, Commune B, District C, Province D" → City="District C", Region="Province D"
When only a province/city is known without a sub-unit, City may be left empty and Region filled, or vice versa — do not infer the missing level.
Do not infer a modern administrative successor unless the biography itself establishes the correspondence.

==========
R6. JOB EXTRACTION
==========
Extract official or professional positions with dates, location, and organization when supported. This includes government, legislative, party, military/security, diplomatic, academic, business, and other clearly defined professional roles.
An organization-name field may be left empty when the biography establishes an independent professional role (e.g. "Writer", "Consultant") without naming an organization.
Use the most specific formal position stated; do not add a generic label like "Member" or "Participant" when a more specific title (Secretary, Chairman, Director, Commander, Minister, Head, etc.) is given in the same passage.
Create separate Job entries for distinct concurrent positions, even if dates and organization overlap.
Preserve historically or institutionally specific titles as stated rather than replacing them with a generic modern equivalent.
Formal temporary or delegated roles — like Head of Delegation, Chair, Negotiator, Signatory, Founder, or an appointed committee member — may be extracted as Jobs when stated. Mere attendance, giving a speech, or an inspection visit is not a Job unless the biography states a formal role.

6.1 ORGANIZATION AND LOCATION NORMALIZATION
- For government bodies (national/federal or subnational — ministries, governments, legislatures, courts, military/security agencies, regional/municipal administrations): if the body's official name explicitly includes a jurisdiction, remove that jurisdiction from the organization-name field and place it first in the location field instead. Do not infer a headquarters city just because it's typical for that kind of institution. Use department/unit fields for a named subordinate body (committee, bureau, department) if explicitly stated.

Examples (national-level government bodies):
Head of State (President, Monarch, or comparable) of [Country] → Position=preserved title (e.g. "President", "King"), Org_Main_Name=the head-of-state institution as named (e.g. "Presidency", "Monarchy"), Location=["[Country]"]
"Ministry of Defense of [Country X], based in [capital city]" → Org_Main_Name="Ministry of Defense", Location=["Country X"]
Head of Government / Deputy Head of Government of [Country] → Position=preserved title, Org_Main_Name=the national government/cabinet body as named (e.g. "Government", "Council of Ministers", "Cabinet"), Location=["[Country]"]
Head of Government / Deputy Head of Government (no jurisdiction stated in that mention) → Position=preserved title, Org_Main_Name=same as above, Location=[]
Elected member of a national legislature's lower/upper chamber → Position="Member of Parliament" (or the stated equivalent, e.g. "Senator"), Org_Main_Name=the chamber's name, Org_Department=the represented constituency/province if stated.
"Chairman of the Budget Committee, National Assembly of [Country], based in [capital city]" → Position="Chairman", Location=["[Country]"], Org_Main_Name="National Assembly", Org_Department="Budget Committee"
Advisor to the Head of Government, serving that person personally → Position="Advisor", Org_Main_Name="Office of the [Head of Government's title]"
Advisor to the Head of Government, serving the institution rather than the individual → Position="Advisor", Org_Main_Name=the government/cabinet body as named, Org_Department="" or named department if stated
Advisor to the Head of State, serving the institution (e.g. Presidency, Monarchy) → Position="Advisor", Org_Main_Name=the head-of-state institution as named

Example (diplomatic representation roles): Ambassador of [Country] to Country X → Position="Ambassador", Org_Main_Name="Ministry of Foreign Affairs", Org_Department="[Country adjective] Embassy", Location=["Country X"].

Examples (subnational public authority — provincial/municipal/district/commune administration or comparable local body):
"Province X Provincial Administration" → Org_Main_Name="Provincial Administration", Location=["Province X"]
"Local Administration of Sub-district Y, City Z" → Org_Main_Name="Local Administration", Location=["Sub-district Y", "City Z"]
"Governor of Province X" → Org_Main_Name="Provincial Administration", Location=["Province X"]

- For non-government organizations (schools, universities, hospitals, businesses, media organizations, political parties/movements, state-owned enterprises): do NOT strip geographic names out of these organizations' official names — the location is normally part of the proper name. Separately record a supported operating location if the text gives one.

Examples (non-government organizations):
"[City] University" → Org_Main_Name="[City] University", Location=["[City]"] only if separately supported as an operating location
"[Country] People's Party" → Org_Main_Name="[Country] People's Party" (no location stripped), Location=["[Country]"]

- Historical jurisdictions (a predecessor state, former union, colonial administration, or defunct governing body): preserve a historical state, predecessor country, or former administrative name only when it is explicit in the organization's name, title, or role description at the time. Do not silently update it to a modern successor.
Example: "Minister of Health, [Former State Name]" → Location=["[Former State Name]"] only if the biography frames it as a jurisdiction context, not merely a regime name attached to a coalition body.

- Formally named coalition, resistance, or transitional governing bodies: a formally named coalition or resistance government/front with its own distinct name is itself Org_Main_Name — do not fold it into a generic "Government" or the country's name.

Example: a coalition body formally named "[Coalition Government Name]" → Org_Main_Name="[Coalition Government Name]"; a ministry or office operating under it (e.g. "Deputy Prime Minister, [Coalition Government Name]") → Org_Main_Name=the coalition body itself, Position=the stated title. Do not upgrade a historical coalition/regime-era role to a modern equivalent institution.

6.2 MILITARY JOBS
Use the service branch (Army, Navy, Air Force, Armed Forces, etc.) as the main organization. Place military districts/zones/regions, commands, theaters, and deployment countries in Location, not Org_Department — unless both a district/zone and a separate geographic location are stated, in which case the district/zone goes to Org_Department and the location clue to Location.

6.2.1 Ordinary military role → Org_Main_Name=the explicitly stated principal force/army as named in that historical period, using the branch name and national adjective actually used in the biography. Common patterns include (but are not limited to):
* "[Country] Armed Forces" (e.g. "Russian Armed Forces") — a unified national force
* "[Country] Army" / "[Country] Navy" / "[Country] Air Force" (e.g. "Vietnamese Navy", "Cambodian Air Force") when a specific branch is stated
* A historically named predecessor force distinct from the current national force (e.g. a former republic's or former regime's army) — preserve the name as stated for that period; do not upgrade it to the modern successor force's name
* A named non-state or resistance armed force explicitly identified in the biography — preserve its stated name as Org_Main_Name

6.2.2 Use Org_Department/Org_Unit for named formations
Preserve formations in descending order from larger to smaller: military region, corps, division, regiment, battalion, company, platoon/squad. Preserve unit numbers and stated honorific/formal names, or source abbreviations such as F9, E1, D3, and C12 when used as unit identifiers

Identify the formation in which the position was directly held and divide the stated organizational chain between Org_Department and Org_Unit based on the following rules:
When one level is stated: Org_Department=division-or-larger; Org_Unit=regiment-or-smaller.
When two subordinate levels are stated: larger=Org_Department; smaller=Org_Unit. Example: "Deputy Commander of Division 9, Corps 4" → Org_Department="Corps 4", Org_Unit="Division 9".
When three or more levels are stated: Org_Department=division-or-larger (levels separated by comma ','); Org_Unit=regiment-or-smaller (levels separated by comma ',').
Example: "Political Commissar of Regiment 1, Division 9, Corps 4" → Org_Department="Corps 4, Division 9", Org_Unit="Regiment 1".
Example: "Political Officer of Company 6, Battalion 2, Regiment 1, Division 9" → Org_Department="Division 9", Org_Unit="Regiment 1, Battalion 2, Company 6".

Within each field, separate formations with commas and preserve descending order. Include only formations explicitly stated for that role; never infer a superior formation or hierarchy from another role.

6.2.3 Consistency
Distinct concurrent military/command positions → separate Job entries.
Military regions/border areas → Location; named forces/armies/fronts/divisions/regiments/units → organization fields.

==========
R7. CAREER-ENDING AND ADVERSE EVENTS
==========
Distinguish by what the biography actually describes, not by assumption:
- Retirement: the end of an entire active career, not merely one position while the person continued working elsewhere.
- Resignation: a voluntary or pressured departure still framed as resignation.
- Dismissal: forced/administrative removal, relief of duties, expulsion from a governing body, or removal through a purge/coup.
- Arrest: actual arrest, detention, house arrest, or imprisonment. Not a mere investigation, accusation, or indictment. A later release, pardon, or acquittal can be noted as detail without changing the event type.
- Exile: forced exile, deportation, expulsion, or compelled flight under explicit threat — NOT voluntary emigration, ordinary relocation, or foreign employment.
- Killed: assassination, execution, extrajudicial killing, or intentionally inflicted death — NOT natural death, illness, or accident.

Multiple event types: When a single episode (e.g. a coup or purge) produces multiple distinct event types (e.g. both dismissal and arrest or both dismissal and exile), create separate entries of corresponding event types, and give them the same Date.

Detail fields should be complete sentences or full bullet points exactly copied/drawn from the biography, kept as separate list items when the source presents them separately — do not merge multiple distinct sentences into one string. Exact dates may remain in Detail even though Date is normalized separately.
Example (correct): ["Dismissed as Mayor following policy failures.", "The Interior Minister oversaw the removal."] — not one merged string.
Example (Correct): ["2007-07-05: retired from being Inspector General; the decision was signed by President A on the same day.", "The retirement was due to health reason."]
Example (Incorrect): ["2007-07-05: retired from being Inspector General; the decision was signed by President A on the same day. The retirement was due to health reason."]

==========
R8. FAMILY MEMBERS
==========
Extract only named family members — do not extract unnamed references (e.g. "his wife", "two sons") without a name.

Use the relative's stated principal name; full-name completion is allowed only when the identity is clear and directly supported by context.

Do not mechanically alter a relative's surname to match the subject's — preserve gendered or otherwise distinct surname forms as stated.

Use specific relationship labels when supported: Wife, Ex-Wife, Husband, Ex-Husband, Partner, Son, Daughter, Child, Father, Mother, Adoptive Father, Adoptive Mother, Stepfather, Stepmother, Half-brother, Half-sister, Brother, Sister, Adoptive Brother, Adoptive Sister, Grandfather, Grandmother, Grandson, Granddaughter, Uncle, Aunt, Nephew, Niece, Relative, Father-in-law, Mother-in-law, Son-in-law, Daughter-in-law, Brother-in-law, Sister-in-law, In-law.

Reason correctly about relationship direction. Relationship direction:
If X is the subject and X's daughter is married to A → A is X's Son-in-law.
If X is married to A and A's father is B → B is X's Father-in-law.
If X's sister A has a son B → B is X's Nephew.
If X's child married B and B is the child of C:
→ B is X's Son-in-law or Daughter-in-law.
→ C may be X's In-law when explicitly relevant and supported.

==========
R9. COLLEAGUE AND RIVAL
==========
9.1 Colleague
Extract a person as a colleague when the biography establishes an explicit, meaningful professional association — working together, collaboration, mentorship, sponsorship/nomination, close political/factional alliance, or comparable explicit relationship. A superior may count as a colleague when a meaningful working/mentorship/alliance relationship is shown, not merely because they formally appointed the subject.
Do not extract a colleague relationship merely from co-occurrence in the same passage, holding a similar office, succession, or a person being quoted about the subject.

Examples:
"Assistant to X between 1980 and 1985" → X is Colleague, FirstMention_Year="1980".
"Founding member of [Movement/Party] alongside [Person A], [Person B], ..." (arriving together as a group in a dated founding episode) → each named person may be Colleague with FirstMention_Year="[founding year]".

9.2 Rival
Extract a person as a rival for an explicit political, professional, factional, or power struggle — including explicitly stated rivalry, factional conflict, a struggle for institutional control, a purge, a coup, or an active effort to remove/arrest/exile/harm the subject's political position.

A person who formally executed or oversaw a routine dismissal/arrest is not automatically a rival — only extract them as Rival if the biography shows they personally organized, plotted, or drove the adverse action.

Organizer/leader of a coup or forced removal against the subject → may be Rival.
Example: "Ousted in a coup led by [Person A] and [Person B]" → both are Rival with FirstMention_Year matching the coup year.

Do not infer rivalry merely from criticism, ordinary policy disagreement, journalism, prosecutorial/judicial roles, electoral competition, or simply being in an opposing party.

9.3 Relationship year and grouping
FirstMention_Year = earliest year stated for that specific relationship, not the first year the person appears anywhere in the biography. Use FirstMention_Year="" when the relationship is explicit but undated. Do not infer the relationship year from general career chronology.

The same person may appear as both colleague and rival at different times if the biography documents a change in the relationship — keep both records rather than deleting the earlier one.

Group several names in one Colleague or Rival entry only when they share (1) the same confirmed FirstMention_Year AND (2) the same event, faction, campaign, or relationship context. Otherwise, create separate entries.
Example: Several founding members of a movement arriving together in a documented founding episode → may be grouped under one shared FirstMention_Year.

Do not extract bare titles (e.g. a royal or religious honorific used alone, "the General," "the Minister") as a person's name when no actual name is given.